



Agilité, DevOps & Sécurité

Cours complet détaillé — support de l'étudiant

Formateur : Haithem Mihoubi

Cursus : 3 modules · 12 jours · 84 heures

Module 1 : Gestion de projet Agile (Scrum & Kanban)

Module 2 : Culture & outils DevOps (CI/CD, Docker, K8s)

Module 3 : Spring Security (JWT, OAuth2, Keycloak)

Niveau : débutant → avancé

Format : cours explicatif complet à lire et à pratiquer

Comment utiliser ce cours

Ce document est un **cours complet** : il explique chaque notion en détail, avec des définitions, des analogies, des exemples concrets et des exercices. Vous pouvez le lire seul, en autonomie. Chaque concept est introduit par **le problème qu'il résout**, puis expliqué, puis illustré.

Conventions de lecture :

-  **Note / Définition** — un point important à mémoriser.
-  **Astuce** — un conseil de pratique.
-  **Attention** — une erreur fréquente à éviter.
-  **Danger** — un point critique (souvent de sécurité).
-  **Exercice / Atelier** — à faire vous-même pour ancrer la notion.

Le cours suit un **fil rouge unique** : une application de commande de repas appelée « **QuickBite** », que l'on conçoit (Module 1), que l'on construit et déploie (Module 2), puis que l'on sécurise (Module 3). Les numéros de page du sommaire ci-après sont **cliquables**.

Sommaire

MODULE 1 — Gestion de projet Agile	4
Chapitre 1 — Pourquoi l'Agilité ? Le problème historique	4
Chapitre 2 — Le Manifeste Agile expliqué en profondeur	6
Chapitre 3 — Prédictif contre adaptatif	8
Chapitre 4 — Les concepts clés	10
Chapitre 5 — Scrum, le cadre complet	12
Chapitre 6 — Écrire et estimer le travail	16
Chapitre 7 — Kanban et le pilotage par le flux	20
Chapitre 8 — Scrum, Kanban ou ScrumBan ? Et les outils	22
MODULE 2 — Culture & outils DevOps	24
Chapitre 9 — Qu'est-ce que le DevOps, vraiment ?	24
Chapitre 10 — Git et le travail collaboratif	26
Chapitre 11 — L'intégration continue (CI) expliquée	27
Chapitre 12 — Docker et la conteneurisation	29
Chapitre 13 — Kubernetes, l'orchestration	33
Chapitre 14 — La chaîne complète : Helm, observabilité, GitOps	37
MODULE 2 · PARTIE PRATIQUE — Linux, Docker, Kubernetes, GitHub & CI/CD	41
Atelier Linux — le système, le terminal et le shell	41
Atelier Docker pratique — de l'image au conteneur	48
Atelier Kubernetes pratique — déployer pour de vrai	52
Atelier GitHub — la plateforme de collaboration	56
Atelier GitHub Actions — CI/CD de bout en bout	58
MODULE 3 — Spring Security	63
Chapitre 15 — Les fondamentaux de la sécurité applicative	63
Chapitre 16 — L'architecture de Spring Security 6	65
Chapitre 17 — Authentification et autorisation (RBAC)	67
Chapitre 18 — Les tokens JWT en détail	70
Chapitre 19 — OAuth2, OpenID Connect et Keycloak	74
Chapitre 20 — Durcissement et bonnes pratiques (OWASP)	77

MODULE 1 — Gestion de projet Agile

Chapitre 1 — Pourquoi l'Agilité ? Le problème historique

1.1 Le contexte : comment on faisait avant

Pour comprendre **pourquoi** l'Agilité existe, il faut comprendre ce qui se passait avant elle. Jusqu'aux années 2000, la quasi-totalité des projets informatiques étaient gérés selon un modèle appelé **cycle en cascade** (en anglais *waterfall*), ou sa variante le **cycle en V**.

L'idée du cycle en cascade est intuitive et rassurante : on traite le projet comme la construction d'un pont. On enchaîne des phases, l'une après l'autre, et on ne passe à la phase suivante que lorsque la précédente est complètement terminée et validée :

1. D'abord, on **recueille tous les besoins** et on rédige un cahier des charges exhaustif.
2. Ensuite, on **conçoit** l'architecture complète du système.
3. Puis on **développe** la totalité du logiciel.
4. Ensuite seulement on **teste** l'ensemble.
5. Enfin, on **livre** au client.

Sur le papier, c'est logique. Le problème, c'est que **le logiciel n'est pas un pont**. Un pont, une fois les besoins exprimés (« relier ces deux rives, supporter tel tonnage »), ne change pas pendant la construction. Un logiciel, lui, évolue constamment : le marché bouge, les concurrents sortent de nouvelles fonctions, les utilisateurs découvrent en s'en servant ce dont ils ont **réellement** besoin — et qui est souvent différent de ce qu'ils avaient demandé au début.

1.2 Les symptômes du problème

Quand un projet dure 12 ou 18 mois et qu'on ne montre le résultat au client qu'à la toute fin, plusieurs catastrophes se produisent de façon récurrente. Les études du secteur (notamment les rapports *CHAOS* du Standish Group) les ont documentées depuis les années 1990 :

- **L'effet tunnel.** Pendant des mois, le client ne voit rien de concret. Il signe un cahier des charges, puis attend. Quand il découvre enfin le produit, il est souvent trop tard pour corriger le tir sans exploser le budget.
- **Des besoins périmés à la livraison.** Ce qui était pertinent il y a 18 mois ne l'est plus forcément. On livre donc, parfois parfaitement, un produit qui ne correspond **plus** au besoin réel.
- **Des fonctionnalités jamais utilisées.** Des analyses ont montré qu'une grande partie des fonctionnalités spécifiées en amont sont **rarement ou jamais utilisées**. On a donc investi énormément d'effort pour produire de la valeur... que personne ne consomme.

- **Le coût du changement explose avec le temps.** Plus on découvre tard qu'une décision était mauvaise, plus elle coûte cher à corriger, car tout ce qui a été construit par-dessus doit être défait.

⚠ Le cœur du problème

Plus l'horizon de livraison est lointain, plus l'écart se creuse entre **ce qui a été spécifié** et **ce dont l'utilisateur a réellement besoin**. L'Agilité ne fait pas disparaître l'incertitude — elle la rend gérable en **raccourcissant le temps entre une idée et sa validation par un vrai utilisateur**.

1.3 La réponse : livrer par petits morceaux et apprendre vite

L'intuition fondatrice de l'Agilité est simple : **si on ne peut pas tout prévoir, alors arrêtons d'essayer de tout prévoir, et organisons-nous plutôt pour apprendre et nous corriger rapidement.**

Concrètement, au lieu de livrer une seule fois après 18 mois, on livre un **petit morceau utilisable** toutes les deux ou trois semaines. À chaque livraison, on montre le résultat à de vrais utilisateurs, on observe leur réaction, et on ajuste la suite. On transforme un grand pari risqué en une série de petits paris peu risqués et corrigeables.

1.4 D'où vient le mot « Agile »

En février 2001, dix-sept experts du développement logiciel se sont réunis dans une station de ski à Snowbird, dans l'Utah. Ils venaient de courants différents — *eXtreme Programming* (Kent Beck), *Scrum* (Ken Schwaber et Jeff Sutherland), *DSDM*, *Crystal* — mais partageaient un même constat : les méthodes lourdes et bureaucratiques étouffaient les projets. De cette réunion est né un texte court : le **Manifeste pour le développement Agile de logiciels**.

■ À retenir absolument

L'Agilité n'est **pas une méthode** avec des étapes à suivre. C'est un **état d'esprit** (un *mindset*) décrit par un manifeste. Ce mindset se décline ensuite en **cadres de travail** (Scrum, Kanban, XP, SAFe...) et en **pratiques concrètes** (intégration continue, tests automatisés, revues...). Quand quelqu'un dit « on fait de l'Agile », demandez-lui toujours : « avec quel cadre, et quelles pratiques ? »

1.5 Quelques repères dans le temps

Année	Ce qui se passe
1986	Takeuchi et Nonaka publient un article comparant le développement produit au rugby ; le terme <i>scrum</i> (la mêlée) y apparaît.
1995	Première présentation formelle de Scrum par Schwaber et Sutherland.
1999	Kent Beck publie <i>Extreme Programming Explained</i> .
2001	Rédaction du Manifeste Agile.
2003	David J. Anderson commence à appliquer Kanban (issu de l'industrie Toyota) au logiciel.
2010+	L'Agilité se généralise et se combine avec DevOps ; apparition des cadres de mise à l'échelle (SAFe, LeSS).

Chapitre 2 — Le Manifeste Agile expliqué en profondeur

Le Manifeste tient en quatre **valeurs** et douze **principes**. Beaucoup de gens les récitent sans les comprendre. Nous allons les expliquer un par un.

2.1 Les quatre valeurs, décortiquées

Chaque valeur s'écrit sous la forme « **A plutôt que B** ». Le point le plus important, et le plus souvent mal compris, est ceci : **B n'est pas mauvais. B a de la valeur. Mais quand il faut choisir, on privilégie A.** Le Manifeste le dit lui-même explicitement.

Valeur 1 — Les individus et leurs interactions, plutôt que les processus et les outils. Un processus bien rodé et un outil sophistiqué ne sauveront jamais une équipe dont les membres ne se parlent pas et ne se font pas confiance. À l'inverse, une équipe soudée trouvera toujours une solution, même avec des outils modestes. *Cela ne veut pas dire* « pas de processus ni d'outils » : *cela veut dire que l'humain passe d'abord.*

Valeur 2 — Un logiciel qui fonctionne, plutôt qu'une documentation exhaustive. La vraie mesure de l'avancement d'un projet, ce n'est pas le nombre de pages de spécifications produites, ni le pourcentage de phases « terminées » sur un planning. C'est **du logiciel qui marche et qu'on peut montrer**. Un document de 200 pages ne rend service à personne s'il ne se traduit pas en produit utilisable. *Cela ne veut pas dire* « zéro documentation » : *cela veut dire* « juste assez de documentation, au bon moment, et un produit qui fonctionne avant tout ».

Valeur 3 — La collaboration avec le client, plutôt que la négociation contractuelle. Dans l'approche traditionnelle, le client et le fournisseur passent un temps fou à négocier un contrat détaillé, puis à se renvoyer la responsabilité quand quelque chose ne va pas (« ce n'était pas dans le contrat ! »). L'Agilité propose de traiter le client comme un **partenaire**

continu : il participe régulièrement, voit le produit grandir, et oriente les priorités. Le contrat existe toujours, mais la relation prime sur le bras de fer.

Valeur 4 — L'adaptation au changement, plutôt que le suivi d'un plan. On planifie toujours — c'est indispensable. Mais le plan est un **outil de décision**, pas une promesse gravée dans le marbre. Dès qu'on apprend quelque chose de nouveau (un retour utilisateur, un problème technique, une opportunité), on ajuste le plan. Dans le monde traditionnel, changer le plan est vu comme un échec ; en Agile, c'est vu comme une **intelligence**.

! L'erreur la plus répandue sur l'Agilité

« Être agile, ça veut dire qu'on ne planifie pas et qu'on ne documente pas. » **C'est faux, et c'est l'inverse de ce que dit le Manifeste.** Le texte précise que les éléments de droite (processus, documentation, contrat, plan) **ont de la valeur**. L'Agilité déplace simplement le curseur vers l'humain, le produit, la collaboration et l'adaptation — elle ne supprime rien.

2.2 Les douze principes, regroupés pour les comprendre

Réciter douze principes dans le désordre ne sert à rien. Regroupons-les en **quatre grandes idées**.

Idée A — Livrer de la valeur tôt, souvent, et accueillir le changement.

1. Notre priorité est de satisfaire le client en livrant tôt et régulièrement des fonctionnalités à grande valeur.
2. Accueillez positivement les changements de besoins, même tard dans le projet : c'est un avantage compétitif pour le client.
3. Livrez fréquemment un logiciel opérationnel, de quelques semaines à quelques mois, la période la plus courte étant préférable.

L'idée centrale : **la valeur se mesure quand elle est entre les mains de l'utilisateur**, pas quand elle est « codée mais pas livrée ». Et puisqu'on livre souvent, accepter un changement coûte peu.

Idée B — La collaboration humaine est le moteur.

1. Les responsables métier et les développeurs doivent travailler ensemble quotidiennement.
2. Réalisez les projets avec des personnes motivées ; donnez-leur l'environnement et le soutien dont elles ont besoin, et faites-leur confiance.
3. La méthode la plus efficace de transmettre de l'information est une conversation en face à face.

L'idée centrale : on ne pilote pas un projet à coups de documents qui s'échangent par e-mail. On le pilote par la **conversation directe et continue** entre ceux qui savent ce qu'il faut faire (le métier) et ceux qui savent le faire (les développeurs).

Idée C — La mesure de la réussite, c'est le produit qui marche, à un rythme tenable.

1. Un logiciel opérationnel est la principale mesure d'avancement.
2. Les processus agiles encouragent un rythme de développement soutenable : commanditaires, développeurs et utilisateurs devraient pouvoir maintenir indéfiniment un rythme constant.

L'idée centrale : on ne « brûle » pas les équipes par des sprints d'urgence permanents. Un bon rythme est un rythme qu'on peut tenir **pendant des années**, pas pendant trois semaines.

Idée D — L'excellence technique et l'amélioration continue.

1. Une attention continue à l'excellence technique et à une bonne conception renforce l'agilité.
2. La simplicité — c'est-à-dire l'art de maximiser la quantité de travail qu'on ne fait pas — est essentielle.
3. Les meilleures architectures, spécifications et conceptions émergent d'équipes auto-organisées.
4. À intervalles réguliers, l'équipe réfléchit aux moyens de devenir plus efficace, puis ajuste son comportement en conséquence.

L'idée centrale : pour pouvoir changer vite et souvent, le code doit rester **propre et simple** (sinon, chaque changement devient un cauchemar). Et l'équipe doit régulièrement **s'arrêter pour réfléchir à comment mieux travailler** — c'est l'origine de la rétrospective qu'on verra plus loin.

Le principe 10 mérite une explication particulière car il est contre-intuitif : « maximiser la quantité de travail qu'on **ne** fait **pas** ». Cela signifie : la meilleure fonctionnalité est souvent celle qu'on décide de **ne pas** développer parce qu'elle n'apporte pas assez de valeur. Chaque ligne de code écrite est une ligne à maintenir, tester et faire évoluer. Moins on en écrit pour un même résultat, mieux c'est.

Chapitre 3 — Prédicatif contre adaptatif

Maintenant que l'esprit est posé, comparons concrètement les deux grandes familles d'approches.

3.1 L'approche prédictive (cascade / cycle en V)

Besoins → Spécifications → Conception → Codage → Tests → Recette → Livraison
(tout est figé au début ; la valeur n'arrive qu'à la toute fin)

Dans cette approche, on cherche à **tout prévoir au début** (d'où le nom « prédictif »). Elle a de vraies qualités, et il serait faux de la diaboliser :

- Elle donne un **cadre rassurant** avec des jalons contractuels nets.
- Elle convient parfaitement quand le besoin est **stable et parfaitement connu** : par exemple un logiciel embarqué dans un dispositif médical certifié, un calcul de paie soumis à une réglementation précise, ou un système qui doit respecter une norme figée.

Ses faiblesses, on les a vues au chapitre 1 : effet tunnel, feedback tardif, coût du changement très élevé en fin de parcours.

3.2 L'approche adaptative (itérative et incrémentale)

Ici, on construit le produit par **incréments** : à chaque itération (une période courte et fixe), on produit un morceau de produit **réellement utilisable**, on le montre, on recueille du feedback, et on ajuste le plan.

Critère	Approche prédictive (V)	Approche adaptative (Agile)
Nature du besoin	Stable et connu d'avance	Évolutif, incertain
Moment où la valeur est livrée	À la toute fin	À chaque itération
Gestion du changement	Coûteuse, encadrée, mal vue	Attendue et intégrée
Comment on mesure l'avancement	En pourcentage de phases finies	En logiciel qui fonctionne
Quand les risques se révèlent	Tard (souvent trop tard)	Tôt (on peut réagir)

3.3 Une distinction subtile mais essentielle : itératif ≠ incrémental

Ces deux mots sont souvent confondus. L'Agilité combine les deux.

- **Incrémental** veut dire : on ajoute des morceaux les uns après les autres. Image : on construit une maison pièce par pièce — d'abord la cuisine entièrement finie, puis le salon entièrement fini, etc.
- **Itératif** veut dire : on repasse plusieurs fois sur l'ensemble pour l'améliorer. Image : on peint d'abord une esquisse grossière du tableau entier, puis on raffine progressivement les détails.

L'illustration du skateboard

La célèbre illustration de Henrik Kniberg résume tout. Pour livrer une voiture, l'approche prédictive construit d'abord une roue (inutilisable), puis deux roues (inutilisable), puis un châssis (inutilisable), et enfin la voiture. L'approche agile livre d'abord un **skateboard** (le client peut déjà se déplacer !), puis une **trottinette**, puis un **vélo**, puis une **moto**, et enfin la **voiture**. À chaque étape, le client a quelque chose d'**utilisable** et peut donner son avis. C'est cela, combiner itératif et incrémental.

Chapitre 4 — Les concepts clés

Quatre notions reviennent en permanence en Agile. Prenons le temps de bien les comprendre, car tout le reste s'appuie dessus.

4.1 Le MVP — Minimum Viable Product (produit minimum viable)

Le MVP est la **plus petite version livrable du produit qui permet déjà d'apprendre quelque chose d'utile auprès de vrais utilisateurs**. Le mot important est « viable » : ce n'est pas un produit bâclé ou cassé, c'est un produit **réduit mais cohérent et fonctionnel**.

Prenons notre fil rouge, l'application QuickBite (commande de repas) :

-  **Mauvais MVP** : « on livre la moitié de l'écran de paiement, sans que ça fonctionne ». Ce n'est pas viable, on n'apprend rien.
-  **Bon MVP** : « on permet de commander **un** plat dans **un** seul restaurant et de payer — du début à la fin ». C'est minimal (un seul restaurant, un seul plat), mais c'est **viable** : un utilisateur peut réellement passer une commande, et on apprend la chose la plus importante : *est-ce que les gens commandent vraiment ?*

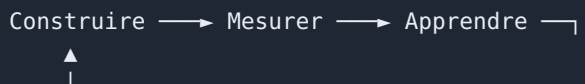
L'objectif du MVP n'est pas de vendre, c'est d'**apprendre** au moindre coût avant d'investir massivement.

4.2 La valeur métier

La **valeur métier** est le bénéfice qu'apporte une fonctionnalité, soit à l'utilisateur, soit à l'entreprise. Cela peut être : générer du revenu, faire économiser du temps ou de l'argent, réduire un risque, ou augmenter la satisfaction des clients.

Le principe fondamental qui en découle : **on priorise par la valeur, pas par la facilité technique**. Il est tentant pour une équipe de faire d'abord ce qui est facile ou amusant à coder. L'Agilité impose au contraire de faire d'abord ce qui rapporte le plus au client, même si c'est plus difficile.

4.3 La boucle de feedback (feedback loop)



C'est le cœur battant de l'Agilité. On construit quelque chose, on **mesure** comment les utilisateurs y réagissent, on en **tire des enseignements**, et ces enseignements alimentent la prochaine construction.

La règle d'or : **plus cette boucle est courte, mieux c'est**. Pourquoi ? Parce que plus on découvre tôt qu'on s'est trompé, moins la correction coûte cher. Découvrir une erreur après deux semaines coûte une correction de deux semaines ; la découvrir après dix-huit mois peut coûter le projet entier.

4.4 Definition of Ready (DoR) et Definition of Done (DoD)

Ce sont deux **listes de critères partagées** par toute l'équipe. Elles évitent les malentendus, qui sont la première cause de gaspillage dans un projet.

	Definition of Ready (DoR)	Definition of Done (DoD)
À quel moment ?	Avant qu'un travail entre dans une itération	Pour déclarer un travail réellement terminé
La question posée	« Cette tâche est-elle prête à être développée ? »	« Ce travail est-il vraiment fini, sans réserve ? »
Exemples de critères	Le besoin est clair, estimé, ses critères d'acceptation sont écrits, ses dépendances sont connues	Le code est écrit, relu par un pair, testé, intégré, documenté, déployable et démontré

⚠ Pourquoi la DoD est non négociable

Le syndrome le plus destructeur en gestion de projet est le « c'est fini à 90 % » qui dure des semaines. Sans une **Definition of Done** claire et partagée, chacun a sa propre idée de « terminé », et le « presque fini » s'accumule jusqu'à ce que plus personne ne sache où en est réellement le projet. La DoD est une **checklist commune et non négociable** : un travail est « Done » seulement s'il coche **toutes** les cases.

Exercice — Atelier 1 : diagnostic et transition

Contexte : « FidExpress » est une PME qui développe un logiciel de fidélité en cascade. Cycle de 12 mois, retards à répétition, clients mécontents, spécifications déjà périmées au moment de la livraison.

À vous :

1. Listez les **symptômes** visibles, puis remontez aux **causes racines** (technique des « 5 pourquoi » : pour chaque problème, demandez « pourquoi ? » cinq fois de suite).
2. Pour trois causes, proposez une **pratique agile** qui y répond directement.
3. Dessinez une **trajectoire de transition** en 3 étapes sur 6 mois (commencez par une seule équipe pilote, puis généralisez).

Éléments de correction de l'exercice :

Symptôme observé	Cause racine probable	Pratique agile qui y répond	Bénéfice attendu
Specs périmées à la livraison	Cycle de livraison trop long	Itérations de 2-3 semaines avec démo	Feedback tôt, specs qui restent vivantes
Effet tunnel, client dans le flou	Aucun point d'étape produit	Revue de sprint régulière	Visibilité continue pour le client
Les mêmes erreurs se répètent	Aucun bilan organisé	Rétrospective d'équipe	Amélioration continue mesurable
Équipe surchargée, burn-out	Plan irréaliste, tout est urgent	Rythme soutenable + limite de travail en cours	Cadence tenable dans la durée

Chapitre 5 — Scrum, le cadre complet

Scrum est, de loin, le cadre agile le plus utilisé dans le monde. Ce n'est pas une méthode rigide : c'est un **cadre léger** qui définit quelques rôles, quelques réunions et quelques documents, et laisse l'équipe libre du reste.

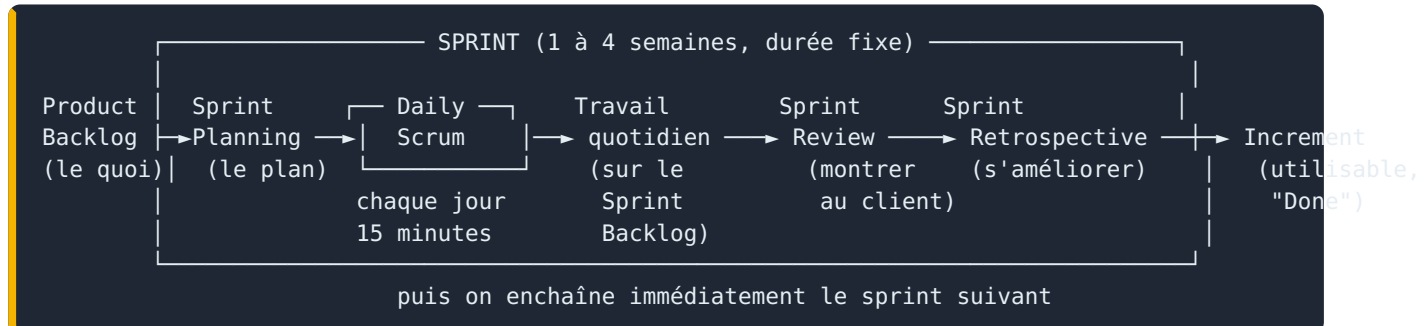
5.1 Le fondement : l'empirisme

Scrum repose sur une idée philosophique appelée **empirisme** : *on ne peut pas tout savoir d'avance ; la connaissance vient de l'expérience et des décisions prises à partir de ce qu'on observe*. Cet empirisme s'appuie sur trois piliers :

- **La transparence.** Tout le monde (équipe, client, management) voit le **même état réel** du travail. Pas de chiffres maquillés, pas de « tout va bien » de façade. Sans transparence, les deux piliers suivants sont impossibles.
- **L'inspection.** On examine régulièrement et fréquemment l'avancement du produit **et** la façon de travailler, pour détecter les écarts.
- **L'adaptation.** Dès qu'on détecte un écart, on **ajuste** — le produit, le plan, ou la manière de travailler — sans attendre.

À ces trois piliers s'ajoutent **cinq valeurs** que l'équipe doit incarner : l'**engagement**, le **focus** (la concentration sur l'objectif), l'**ouverture**, le **respect** et le **courage** (notamment le courage de dire la vérité et de soulever les problèmes).

5.2 Vue d'ensemble du cadre



Décrivons maintenant chaque élément en détail.

5.3 Les trois rôles (ou « responsabilités »)

Scrum définit **une seule équipe**, appelée *Scrum Team*, sans hiérarchie interne. Elle regroupe trois responsabilités distinctes.

Le Product Owner (PO) — il répond à « quoi » et « pourquoi »

Le Product Owner est la personne **responsable de maximiser la valeur** du produit. C'est lui — et lui seul — qui décide de ce qu'on fait et dans quel ordre.

Ses responsabilités concrètes :

- **Porter la vision du produit** : où va-t-on, et pourquoi ?
- **Constituer, ordonner et clarifier le Product Backlog** (la liste de tout ce qu'on pourrait faire — voir plus loin).
- **Prioriser** : décider quoi faire en premier, en fonction de la valeur, du risque, du coût et des dépendances.
- **Être disponible** pour l'équipe : répondre aux questions, clarifier les besoins, valider ou refuser le résultat.

Le PO utilise des outils comme le **story mapping** (cartographier le parcours utilisateur pour organiser le backlog) et des techniques de priorisation comme **MoSCoW** ou **valeur/effort** (qu'on verra au chapitre 6).

💡 Le PO décrit le besoin, pas la solution

Un bon Product Owner exprime un **besoin** et un **pourquoi**, puis laisse l'équipe trouver le **comment**. Il dit : « j'ai besoin que le client puisse payer en un minimum d'étapes, car on perd des ventes au moment du paiement ». Il ne dit **pas** : « ajoutez un bouton bleu de 40 pixels en bas à droite ». Le « comment » technique appartient à l'équipe.

Le Scrum Master (SM) — il répond à « comment bien travailler ensemble »

Le Scrum Master est un **leader serviteur** (*servant leader*). Cette expression est importante : il ne **commande** pas l'équipe, il la **sert**. Son but est de rendre l'équipe la plus efficace possible.

Ses responsabilités concrètes :

- **Faciliter** les réunions Scrum et fluidifier la collaboration.
- **Lever les obstacles** (en anglais *impediments*) : tout ce qui ralentit l'équipe (un accès qu'on n'obtient pas, un autre service qui ne répond pas, un outil qui manque). Le SM passe une grande partie de son temps à débloquer ces situations.
- **Coach** l'équipe vers l'auto-organisation et l'amélioration continue.
- **Protéger** l'équipe des interruptions extérieures et du sur-engagement (par exemple, dire non à un manager qui veut ajouter du travail en plein milieu d'un sprint).

⚠ Un Scrum Master n'est PAS un chef de projet

C'est la confusion la plus fréquente. Le Scrum Master ne distribue pas les tâches, ne fixe pas les délais à la place de l'équipe, et n'évalue pas individuellement les personnes. Son seul pouvoir est l'**influence** et le **coaching**, pas l'**autorité hiérarchique**. S'il se met à donner des ordres, ce n'est plus du Scrum.

Les Développeurs — ils répondent à « comment réaliser »

Les *Developers* regroupent toutes les personnes qui contribuent à construire l'incrément : développeurs au sens strict, mais aussi testeurs, designers, experts ops, etc. Deux qualités les définissent :

- Ils sont **auto-organisés** : c'est l'équipe elle-même qui décide comment se répartir et réaliser le travail, pas un chef extérieur.
- Ils sont **pluridisciplinaires** (*cross-functional*) : l'équipe possède collectivement toutes les compétences nécessaires pour livrer, sans dépendre en permanence de l'extérieur.

	Product Owner	Scrum Master	Développeurs
Se concentre sur...	La valeur	Le bon fonctionnement de l'équipe	La réalisation
Est responsable de...	Le Product Backlog	Le processus Scrum	Le Sprint Backlog
Sa question type	« Quoi faire, et dans quel ordre ? »	« Qu'est-ce qui nous bloque ? »	« Comment le faire, et combien ça coûte ? »

5.4 Les trois artefacts

Un **artefact**, en Scrum, est un élément concret qui rend le travail visible. Il y en a trois, et à chacun est associé un « engagement » qui lui donne une direction.

Artefact	Ce que c'est	Engagement associé
Product Backlog	La liste ordonnée et vivante de tout ce qui pourrait être utile au produit. C'est la source unique du travail à faire.	Product Goal : l'objectif produit à moyen terme
Sprint Backlog	La sélection d'éléments choisis pour le sprint en cours, plus le plan pour les réaliser.	Sprint Goal : l'objectif unique du sprint
Increment	La somme de tout le travail « Done » à la fin du sprint. Il doit être utilisable .	Definition of Done : la définition de « terminé »

■ « Ordonné » et non « priorisé »

On dit que le Product Backlog est **ordonné** plutôt que **priorisé**. La nuance : « priorisé » pourrait laisser croire qu'il suffit de classer par importance. « Ordonné » est plus fort : il existe un **ordre précis, du premier au dernier élément**, qui tient compte de la valeur mais aussi du risque, des dépendances et du coût. Il n'y a jamais deux éléments « ex æquo » : l'équipe sait toujours quoi prendre ensuite.

5.5 Les événements (les cérémonies)

Scrum définit cinq événements. Le premier, le **Sprint**, contient tous les autres.

Le Sprint est le conteneur : une période de durée **fixe**, de une à quatre semaines (deux semaines est le choix le plus courant). On n'allonge jamais un sprint, même si le travail n'est pas fini. Dès qu'un sprint se termine, le suivant commence immédiatement. Cette régularité crée un **rythme** (une « cadence ») rassurant et prévisible.

Événement	Durée maximale (sprint de 2 semaines)	Qui participe	Objectif
Sprint Planning	4 heures	Toute l'équipe Scrum	Définir l'objectif du sprint et le plan
Daily Scrum	15 minutes par jour	Les Développeurs	Se synchroniser et ajuster le plan du jour
Sprint Review	2 heures	L'équipe + les parties prenantes	Inspecter le produit et recueillir le feedback
Sprint Retrospective	1 h 30	Toute l'équipe Scrum	Améliorer la façon de travailler

Le Sprint Planning — la réunion qui lance le sprint

Au début de chaque sprint, l'équipe se réunit pour planifier. Cette réunion répond à trois questions, dans l'ordre :

1. **Pourquoi ce sprint a-t-il de la valeur ?** On formule un **Sprint Goal** (objectif de sprint) : une phrase qui donne du sens à l'ensemble. Exemple pour QuickBite : « permettre à un client de commander un plat et de payer, de bout en bout ».

2. **Que peut-on livrer dans ce sprint ?** L'équipe sélectionne, en haut du Product Backlog, les éléments qu'elle pense pouvoir terminer.
3. **Comment va-t-on les réaliser ?** Les développeurs découpent ces éléments en tâches concrètes : c'est le **plan**.

Le Daily Scrum — la synchronisation quotidienne

Chaque jour, à heure fixe et pour **15 minutes maximum**, les développeurs se synchronisent. Un format classique consiste à ce que chacun réponde brièvement à trois questions : *qu'ai-je terminé hier qui aide l'objectif du sprint ? que vais-je faire aujourd'hui ? qu'est-ce qui me bloque ?*

⚠ Le Daily n'est pas un reporting au chef

Le Daily Scrum est une réunion **entre développeurs, pour les développeurs**. Ce n'est pas un compte-rendu fait au Scrum Master ou au manager. Si un sujet demande une discussion approfondie, on le note et on le traite **juste après**, entre les seules personnes concernées (on appelle cela « l'after-party »). C'est ce qui permet de tenir les 15 minutes.

La Sprint Review — montrer le produit

À la fin du sprint, l'équipe présente l'incrément réalisé **aux parties prenantes** (le client, les utilisateurs, le management). On fait une **démonstration réelle** du produit qui fonctionne (pas des diapositives !), et on recueille du feedback qui alimentera le Product Backlog. La Review inspecte **le produit**.

La Sprint Retrospective — améliorer la façon de travailler

Juste après la Review, l'équipe se réunit entre elle pour réfléchir non pas au produit, mais à **sa façon de travailler** : qu'est-ce qui a bien marché ? qu'est-ce qui nous a ralenti ? que va-t-on changer concrètement au prochain sprint ? La Rétrospective inspecte **le processus**.

💡 Review et Rétrospective : ne pas confondre

Moyen mnémotechnique : la **Review** regarde le **pROduit** (et inclut le client) ; la **Rétro** regarde l'équi**PE** et sa façon de travailler (entre membres de l'équipe uniquement). Quelques formats de rétrospective utiles : **Start / Stop / Continue** (qu'est-ce qu'on commence, arrête, continue ?), **Mad / Sad / Glad** (ce qui m'a énervé, attristé, réjoui), et les **4 L** (Liked, Learned, Lacked, Longed for).

Chapitre 6 — Écrire et estimer le travail

On sait maintenant qui fait quoi et quand. Reste une question pratique : **comment décrit-on concrètement le travail à faire ?**

6.1 La hiérarchie : Epic → User Story → Task

Epic (un gros besoin, trop gros pour un seul sprint)
└─ User Story (un besoin utilisateur livrable en un sprint)
 └─ Task (un travail technique de quelques heures)

- Une **Epic** est un gros morceau de fonctionnalité qu'on ne peut pas réaliser en un seul sprint (exemple : « tout le système de paiement »). On la découpe en plusieurs User Stories.
- Une **User Story** (« récit utilisateur ») décrit un besoin **du point de vue de l'utilisateur**, suffisamment petit pour tenir dans un sprint.
- Une **Task** est une décomposition technique d'une story (exemple : « créer la table en base », « appeler l'API de paiement »). Elle se compte en heures.

6.2 Comment écrire une bonne User Story

La forme canonique d'une story est une phrase qui force à penser à l'utilisateur et au bénéfice :

En tant que [rôle], **je veux** [action], **afin de** [bénéfice].

Exemple pour QuickBite :

En tant que **client**, je veux **payer ma commande par carte bancaire**, afin de **finaliser mon achat sans quitter l'application**.

Cette formulation est précieuse car elle répond toujours à « pour qui ? » et surtout à « pourquoi ? ». Le « afin de » est la partie la plus importante : si on n'arrive pas à le remplir, c'est peut-être que la fonctionnalité n'a pas de valeur réelle.

6.3 Les critères INVEST : la check-list d'une bonne story

Pour vérifier qu'une story est bien écrite, on utilise l'acronyme **INVEST** :

Lettre	Signifie	La question à se poser
I	<i>Independent</i> (indépendante)	Peut-on la réaliser seule, sans dépendre d'une autre ?
N	<i>Negotiable</i> (négociable)	Le « comment » reste-t-il ouvert à la discussion ?
V	<i>Valuable</i> (à valeur)	Apporte-t-elle un bénéfice clair à quelqu'un ?
E	<i>Estimable</i> (estimable)	A-t-on assez d'infos pour estimer son effort ?
S	<i>Small</i> (petite)	Tient-elle dans un seul sprint ?
T	<i>Testable</i> (testable)	Peut-on vérifier objectivement qu'elle est réussie ?

6.4 Les critères d'acceptation

Une story doit s'accompagner de **critères d'acceptation** : les conditions précises qui permettent de dire « c'est réussi ». Un format très répandu est le format **Gherkin** (Étant donné... Quand... Alors) :

```
Étant donné que je suis un client avec un panier non vide
Quand je saisis une carte bancaire valide et que je confirme
Alors le paiement est accepté
Et je reçois un récapitulatif de ma commande
```

Ces critères servent à trois choses : ils clarifient le besoin (le « T » d'INVEST), ils guident le développement, et ils servent de base aux tests.

6.5 Estimer : pourquoi en « points » et pas en heures

Voici un point qui surprend souvent les débutants : en Agile, on n'estime généralement **pas en heures**, mais en **points de story** (*story points*), en utilisant une suite de nombres inspirée de Fibonacci : 1, 2, 3, 5, 8, 13, 20...

Pourquoi pas en heures ? Parce que les estimations en heures donnent une **fausse impression de précision** et se révèlent presque toujours fausses (« je pensais 4 heures, ça en a pris 11 »). Les humains sont mauvais pour estimer des durées absolues, mais **bons pour comparer** : on sait facilement dire qu'une tâche est « à peu près deux fois plus grosse » qu'une autre.

Les points capturent donc la **taille relative** (complexité + effort + incertitude) d'une story par rapport aux autres. On choisit une petite story de référence (« celle-ci vaut 2 »), et on estime tout le reste par comparaison.

■ La vélocité : prévoir sans promettre des heures

Une fois qu'on estime en points, on observe combien de points l'équipe termine réellement à chaque sprint : c'est la **vélocité**. Si une équipe livre en moyenne 20 points par sprint, et qu'il reste 100 points dans le backlog, on peut estimer qu'il faudra environ 5 sprints. C'est une prévision **basée sur des faits observés**, bien plus fiable qu'une addition d'heures devinées.

Le Planning Poker : comment on estime en équipe

Pour estimer collectivement et éviter qu'une personne influence les autres, on utilise le **Planning Poker** :

1. Le Product Owner présente une story.
2. Chaque membre de l'équipe choisit **en secret** une carte (un nombre de la suite).
3. Tout le monde **révèle sa carte en même temps**.
4. Si les estimations divergent fortement, les deux extrêmes **expliquent leur raisonnement** (souvent, l'un a vu une difficulté que l'autre ignorait). Puis on rejoue, jusqu'au consensus.

L'intérêt n'est pas tant le chiffre final que **la conversation** qu'il provoque : elle révèle les incompréhensions et les risques cachés.

6.6 Prioriser avec MoSCoW

Pour ordonner le backlog, une technique simple est **MoSCoW**, qui classe chaque élément en quatre catégories :

- **Must** (doit) — indispensable ; sans cela, le produit n'a pas de sens. C'est le cœur du MVP.
- **Should** (devrait) — important, mais on peut s'en passer temporairement.
- **Could** (pourrait) — souhaitable si on a le temps.
- **Won't** (pas maintenant) — explicitement hors périmètre pour l'instant.

Exercice — Atelier 2 : construire un backlog et planifier un sprint

Produit : QuickBite. Vision : « permettre à un client de commander un repas dans un restaurant proche et de payer en quelques clics ».

À vous :

1. **Brainstormez** au moins 12 idées d'éléments de backlog.
2. Écrivez **6 User Stories** au format canonique, avec des critères d'acceptation Gherkin pour deux d'entre elles.
3. **Estimez**-les en Planning Poker (suite de Fibonacci).
4. **Priorisez** avec MoSCoW et formulez le **Product Goal**.
5. La capacité de l'équipe est de **20 points** : choisissez les stories du Sprint 1 et formulez un **Sprint Goal**.

Exemple de correction (extrait du backlog estimé et priorisé) :

#	User Story	Estimation	MoSCoW
1	Voir la liste des restaurants proches	5	Must
2	Consulter le menu d'un restaurant	3	Must
3	Ajouter un plat au panier	3	Must
4	Payer ma commande par carte	8	Must
5	Suivre l'état de ma commande	5	Should
6	Noter le restaurant après livraison	2	Could

Sprint Goal proposé : « Un client peut commander un plat dans un restaurant et le payer — parcours complet de bout en bout. » On sélectionne alors les stories 1, 2, 3 et 4, soit 19 points : cela tient dans la capacité de 20 points, et l'ensemble forme un objectif **cohérent** (et pas une simple addition de tâches sans lien).

Chapitre 7 — Kanban et le pilotage par le flux

Scrum n'est pas le seul cadre agile. **Kanban** en est un autre, très différent dans sa philosophie, et tout aussi important à connaître.

7.1 D'où vient Kanban

Le mot *kanban* signifie « panneau » ou « carte visuelle » en japonais. La méthode vient du **système de production de Toyota** (le fameux « juste-à-temps »), où des cartes physiques circulaient pour signaler qu'il fallait réapprovisionner un poste. David J. Anderson a adapté ces idées au développement logiciel à partir de 2010.

La grande différence avec Scrum : **Kanban n'impose ni rôles, ni itérations, ni réunions obligatoires**. C'est une **méthode d'amélioration évolutive** : on part de la façon dont on travaille **déjà aujourd'hui**, et on l'améliore progressivement, sans révolution.

7.2 Les principes de conduite du changement

Kanban repose sur quatre principes pour démarrer en douceur :

1. **Commencer par ce que vous faites maintenant** — on ne jette rien, on observe l'existant.
2. **S'accorder pour progresser par des changements évolutifs et incrémentaux** — pas de grand chambardement.
3. **Respecter le processus, les rôles et les responsabilités actuels** — on ne supprime pas les titres ni les fonctions des gens.
4. **Encourager le leadership à tous les niveaux** — les bonnes idées d'amélioration peuvent venir de n'importe qui.

7.3 Les pratiques fondamentales

Kanban se résume à six pratiques. Les deux premières sont les plus importantes.

Pratique 1 — Visualiser le travail. On matérialise tout le travail sur un **tableau** (le tableau Kanban) divisé en colonnes représentant les étapes du flux. Chaque tâche est une carte qui se déplace de gauche à droite.

À FAIRE	EN COURS (3)	REVUE (2)	TERMINÉ
[C7] [C8] [C9]	[C3] [C4] [C5]	[C2]	[C1]

▲ limite = 3

▲ limite = 2

Le simple fait de **rendre le travail visible** change tout : on voit immédiatement où ça coince, ce qui s'accumule, qui est surchargé.

Pratique 2 — Limiter le travail en cours (WIP, Work In Progress). C'est la pratique signature de Kanban, et la plus contre-intuitive. On fixe un **nombre maximum de cartes** autorisées simultanément dans chaque colonne. Dans l'exemple, la colonne « En cours » ne peut pas contenir plus de 3 cartes.

Pourquoi limiter volontairement le travail ? Parce que cela produit trois effets vertueux :

- On est **forcé de terminer** ce qui est en cours avant d'en commencer du nouveau. La devise de Kanban est : « *Stop starting, start finishing* » — arrêtez de commencer, commencez à finir.
- Les **goulots d'étranglement deviennent visibles** : si une colonne est pleine et bloque tout, on voit exactement où est le problème.
- Le **délai de traitement diminue** mécaniquement (on l'explique juste après).

! Le multitâche détruit le débit

Sans limite de WIP, le réflexe humain est de tout commencer en même temps. Résultat : dix choses en cours, zéro terminée, et un client qui attend tout. Limiter le WIP semble ralentir l'équipe (« comment ça, je ne peux pas commencer une nouvelle tâche ?! »), mais c'est en réalité le **levier numéro un** pour livrer plus vite. C'est prouvé par la loi de Little, ci-dessous.

7.4 Les métriques du flux

Kanban se pilote avec des chiffres simples.

Métrique	Définition	À quoi elle sert
Lead time (délai de livraison)	Temps total entre le moment où le client demande quelque chose et le moment où il le reçoit	Prévoir et tenir des délais
Cycle time (temps de cycle)	Temps entre le début effectif du travail et sa fin	Améliorer l'exécution
Throughput (débit)	Nombre de tâches terminées par unité de temps (par semaine, par exemple)	Mesurer la capacité réelle de l'équipe
WIP	Nombre de tâches en cours à un instant donné	Réguler le flux

Ces métriques sont reliées par une formule célèbre, la **loi de Little** :

$$\text{Lead time} \approx \text{WIP} \div \text{Throughput}$$

Lisons-la : si une équipe a 20 tâches en cours (WIP) et en termine 5 par semaine (throughput), le délai moyen est d'environ $20 \div 5 = 4$ **semaines**. Si l'on réduit le WIP à 10 (à débit constant), le délai tombe à $10 \div 5 = 2$ **semaines**. **Voilà pourquoi limiter le WIP accélère la livraison.**

Le diagramme de flux cumulé (CFD)

Le *Cumulative Flow Diagram* est un graphique qui empile, jour après jour, le nombre de tâches dans chaque état. Sa lecture est intuitive : une bande qui **s'élargit** dans le temps signale une **accumulation** (un goulot d'étranglement) ; des bandes **parallèles et fines** signalent un **flux sain** et régulier.

Exercice — Atelier 3 : simuler un flux Kanban

Modélisez le flux du support de QuickBite (colonnes : Backlog → Analyse → Dev → Test → Déployé). Posez des limites de WIP réalistes. Simulez 5 « jours » en faisant entrer 2 cartes par jour et en avançant les cartes selon une capacité par colonne. Notez le délai de chaque carte. Vous verrez très probablement un **goulot** apparaître dans une colonne (souvent « Test »). Baissez alors une limite de WIP en amont, ou renforcez la colonne saturée, et rejouez : observez l'effet sur le délai moyen.

Ce que l'exercice doit révéler : le goulot apparaît là où les cartes s'empilent. La solution n'est presque jamais de « travailler plus vite », mais de **lisser le flux** : réduire ce qui entre en amont, ou aider la colonne saturée (entraide, ce qu'on appelle le *swarming*). La leçon profonde, issue de la *théorie des contraintes*, est qu'**optimiser chaque poste isolément ne sert à rien ; il faut optimiser le flux dans son ensemble**.

Chapitre 8 — Scrum, Kanban ou ScrumBan ? Et les outils

8.1 Le tableau comparatif

Critère	Scrum	Kanban
Rythme	Itérations de durée fixe (sprints)	Flux continu, sans itérations
Rôles	PO, Scrum Master, Développeurs	Aucun rôle imposé
Engagement	Un Sprint Goal par sprint	Pas d'engagement par lot
Limite du travail	La capacité du sprint	Une limite de WIP par colonne
Changement en cours de route	Évité pendant le sprint	Possible à tout moment
Idéal pour...	Développer un nouveau produit	Un flux continu et imprévisible

8.2 Comment choisir

- Choisissez **Scrum** quand vous **développez un produit** et que vous avez besoin de cadence, d'objectifs clairs et d'un travail planifiable par lots. C'est le cas typique d'une équipe qui construit QuickBite.
- Choisissez **Kanban** quand le travail **arrive de façon imprévisible** et qu'on ne peut pas le planifier par sprints : support client, infogérance, exploitation, production de contenu.

- Choisissez **ScrumBan** (un hybride) dans deux cas : soit une équipe Scrum **submergée par les urgences** qui cassent ses sprints, soit une équipe en **transition douce** vers l'agilité. ScrumBan garde certains éléments de Scrum (les rôles, la rétrospective) mais pilote le travail par le **flux et les limites de WIP** plutôt que par des sprints rigides.

💡 Il n'y a pas de « meilleur » cadre

La bonne question n'est jamais « Scrum ou Kanban, lequel est le meilleur ? », mais « **quel cadre correspond à la nature de mon travail ?** ». Une même entreprise utilise souvent Scrum pour ses équipes produit et Kanban pour son support.

8.3 Les outils du marché

Outil	Points forts	Idéal pour
Jira	Très puissant, reporting riche, gère l'agilité à grande échelle	Équipes produit, grandes organisations
Trello	Extrêmement simple, prise en main immédiate	Petites équipes, Kanban léger
ClickUp	Polyvalent (tâches, documents, objectifs, vues multiples)	Équipes aux usages variés
GitHub Projects	Couplé directement au code et aux issues GitHub	Équipes de développeurs
Asana	Bonne gestion des tâches, des échéances et des dépendances	Coordination transverse

🔧 Exercice — créer votre premier board

Sur **Trello** ou **GitHub Projects** (comptes gratuits) : créez le tableau QuickBite, ajoutez les colonnes d'un flux, créez 6 cartes à partir des User Stories de l'atelier 2, ajoutez des étiquettes de couleur pour le classement MoSCoW, et déplacez deux cartes pour simuler l'avancement. Si l'outil le permet, activez une limite de WIP sur la colonne « En cours ».

8.4 Récapitulatif du Module 1

Au terme de ce module, retenez la ligne directrice : **l'Agilité consiste à livrer de la valeur tôt et souvent, à apprendre des retours, et à s'adapter en continu.** Scrum organise cela par des **sprints** avec des rôles et des cérémonies ; Kanban l'organise par un **flux** qu'on visualise et qu'on limite. Dans les deux cas, le but final est identique : raccourcir la boucle entre une idée et sa validation par un vrai utilisateur.

Dans le Module 2, nous verrons **comment livrer techniquement** ce produit, vite et de façon fiable, grâce au DevOps.

MODULE 2 — Culture & outils DevOps

Chapitre 9 — Qu'est-ce que le DevOps, vraiment ?

9.1 Le problème : le « mur de la confusion »

Pour comprendre le DevOps, il faut comprendre le conflit qu'il résout. Dans une organisation informatique classique, il existe deux équipes aux objectifs **opposés** :

- Les **développeurs** (Dev) sont récompensés quand ils livrent du **changement** : de nouvelles fonctionnalités, vite. Leur instinct est de pousser des modifications en permanence.
- Les **opérations** (Ops) — ceux qui exploitent les serveurs en production — sont récompensés quand le système est **stable**. Or, chaque changement est une source potentielle de panne. Leur instinct est donc de **freiner** les changements.

Ces deux équipes se renvoyaient la responsabilité par-dessus un « mur » imaginaire. Le développeur livrait son code en disant « ça marche sur ma machine », et l'opérationnel, quand ça plantait en production, répondait « ce n'est pas un problème de serveur, c'est votre code ». Personne n'était responsable de bout en bout. Les déploiements étaient donc **rares, manuels, stressants et risqués** — on déployait parfois une fois par trimestre, en pleine nuit, en croisant les doigts.

9.2 La réponse : réunir Dev et Ops

Le mouvement **DevOps** (le mot est la contraction de *Development* et *Operations*), popularisé vers 2009 lors des premières conférences *DevOpsDays*, propose de **détruire ce mur**. L'idée : Dev et Ops partagent désormais une **responsabilité commune**, du code jusqu'à la production, soutenue par une forte **automatisation**.

Ce que DevOps n'est pas

Le DevOps n'est **ni un métier** (« je recrute un DevOps » est un abus de langage répandu), **ni un outil** qu'on installerait. C'est avant tout une **culture** et un ensemble de **pratiques** visant à raccourcir le temps entre une idée et sa mise en production, de manière fiable et répétable. Les outils (Docker, Kubernetes, etc.) ne sont que des moyens au service de cette culture.

9.3 Le modèle CAMS : les quatre piliers

On résume la culture DevOps par l'acronyme **CAMS** :

Pilier	Idée	Exemple concret
C — Culture	Collaboration et responsabilité partagée entre Dev et Ops	Les développeurs sont d'astreinte sur leur propre code en production
A — Automation	Automatiser tout ce qui est répétitif et manuel	Le build, les tests et le déploiement se déclenchent tout seuls
M — Measurement	Mesurer pour pouvoir décider et s'améliorer	On suit le délai de livraison, le taux d'échec, le temps de réparation
S — Sharing	Partager le savoir et apprendre des incidents sans blâmer	On rédige des post-mortems « sans coupable » après chaque incident

9.4 Comment mesurer si on fait du « bon » DevOps : les métriques DORA

Un programme de recherche (le *DevOps Research and Assessment*, ou DORA, racheté par Google) a identifié **quatre indicateurs** qui distinguent les équipes très performantes des autres :

1. **Fréquence de déploiement** — à quelle fréquence livre-t-on en production ? (Les meilleures équipes déploient plusieurs fois par jour.)
2. **Délai de livraison d'un changement** (*Lead Time for Changes*) — combien de temps s'écoule entre l'écriture du code et sa mise en production ?
3. **Taux d'échec des changements** (*Change Failure Rate*) — quel pourcentage de déploiements provoque un incident ?
4. **Temps moyen de rétablissement** (*MTTR, Mean Time To Restore*) — quand un incident survient, combien de temps faut-il pour rétablir le service ?

Les deux premières mesurent la **vitesse**, les deux dernières mesurent la **stabilité**. L'enseignement majeur de DORA est contre-intuitif : **vitesse et stabilité ne s'opposent pas**. Les meilleures équipes déploient à la fois plus souvent **et** avec moins de pannes — parce que de petits changements fréquents sont plus faciles à tester et à corriger que de gros changements rares.

9.5 CI, CD et CD : trois marches à ne pas confondre

Trois sigles très proches sèment la confusion. Distinguons-les clairement :

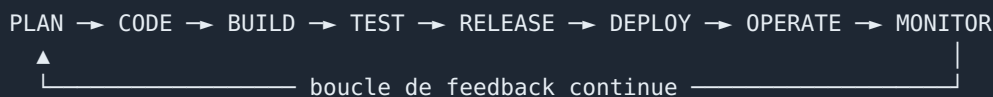
- **CI — Intégration Continue** (*Continuous Integration*) : à **chaque** modification du code, on l'intègre automatiquement au reste et on lance les tests. But : détecter les problèmes **immédiatement**, pas des semaines plus tard.
- **CD — Livraison Continue** (*Continuous Delivery*) : on va plus loin : à tout moment, l'application est dans un état **déployable**. Le déploiement vers la production reste déclenché **manuellement** (par un humain qui appuie sur le bouton).
- **CD — Déploiement Continu** (*Continuous Deployment*) : la dernière marche : le déploiement en production devient lui aussi **automatique**, sans intervention humaine, dès que les tests passent.

💡 On gravit ces marches dans l'ordre

On ne saute pas directement au déploiement continu. On commence par maîtriser l'**intégration continue** (avoir une suite de tests fiable), puis on automatise jusqu'à un produit toujours **déploiable** (livraison continue), et enfin, lorsque la confiance est suffisante, on automatise le déploiement lui-même.

9.6 Le cycle de vie DevOps

On représente souvent le DevOps comme une boucle infinie, car le travail ne s'arrête jamais : ce qu'on apprend en production alimente la planification suivante.



- **Plan / Code** : on planifie (lien direct avec l'Agile du Module 1) et on écrit le code, géré dans Git.
- **Build / Test** : on compile et on lance les tests automatisés, plus des analyses de qualité et de sécurité.
- **Release / Deploy** : on produit un artefact versionné et on le déploie.
- **Operate / Monitor** : on exploite et on surveille le système en production. Les anomalies détectées repartent dans le « Plan ».

Chapitre 10 — Git et le travail collaboratif

Tout commence par le code, et le code se gère avec **Git**, le système de gestion de versions universel. Nous supposons les bases connues ; concentrons-nous sur l'usage collaboratif.

10.1 Les commandes du quotidien

```
git init                # créer un nouveau dépôt local
git clone <url>          # copier un dépôt existant
git switch -c feature/paiement # créer une branche et basculer dessus
git add .               # préparer les fichiers modifiés
git commit -m "feat: ajout du paiement" # enregistrer un instantané
git push -u origin feature/paiement # envoyer la branche sur le serveur
# puis on ouvre une « Pull Request » (PR) ou « Merge Request » (MR)
```

Une **branche** est une ligne de développement parallèle : elle permet de travailler sur une fonctionnalité sans perturber le code principal. Une **Pull Request** est une demande de fusion de votre branche dans la branche principale, accompagnée d'une **revue** par un collègue.

10.2 Bien nommer ses commits : les Conventional Commits

Pour que l'historique soit lisible (et pour automatiser la génération de notes de version), on adopte une convention de préfixes pour les messages de commit :

```
feat:    une nouvelle fonctionnalité
fix:     une correction de bug
docs:    de la documentation
refactor: une refonte du code sans changement de comportement
test:    l'ajout ou la modification de tests
chore:   des tâches techniques (mise à jour de dépendances, configuration)
```

10.3 Les stratégies de branches

Il existe plusieurs manières d'organiser les branches d'un projet. Les trois principales :

Stratégie	Principe	Convient à...
Git Flow	Des branches durables (<code>main</code> , <code>develop</code>) et temporaires (<code>feature/*</code> , <code>release/*</code> , <code>hotfix/*</code>)	Des produits avec des versions planifiées
GitHub Flow	Une seule branche durable (<code>main</code>) + des branches courtes + des PR + un déploiement fréquent	Le web et le SaaS, livraison continue
Trunk-Based	On commit très souvent directement sur <code>main</code> , en cachant les fonctionnalités inachevées derrière des « feature flags »	Les équipes matures avec une CI solide

⚠ Les branches qui vivent trop longtemps font mal

Plus une branche reste isolée longtemps, plus elle diverge du code principal, et plus sa fusion devient un cauchemar de conflits. C'est exactement ce que l'**intégration continue** cherche à éviter : elle encourage des branches **courtes** (quelques jours au maximum) et des fusions **fréquentes**.

Chapitre 11 — L'intégration continue (CI) expliquée

11.1 Ce qu'est un pipeline

Un **pipeline** est une suite d'étapes automatisées (on les appelle *stages*) qui se déclenchent toutes seules en réponse à un événement — typiquement, un `push` de code ou l'ouverture d'une Pull Request. Un pipeline d'intégration continue classique enchaîne :

```
push → [récupérer le code] → [installer les dépendances] → [analyser le style]
      → [compiler] → [lancer les tests] → [produire l'artefact] → (✓ ou ✗)
```

Trois principes guident un bon pipeline :

- **Rapide** : le retour doit arriver en quelques minutes, sinon les développeurs ne l'attendent pas.

- **Reproductible** : il doit s'exécuter de la même façon partout, avec les mêmes versions d'outils. C'est ce qui élimine le « ça marche sur ma machine ».
- **Fail fast** (« échouer vite ») : si une étape échoue, on arrête tout de suite et on prévient. Et la règle d'or : **on ne fusionne jamais une branche dont le pipeline est rouge.**

11.2 Un pipeline concret avec GitHub Actions

Avec GitHub Actions, on décrit le pipeline dans un fichier YAML placé dans le dossier `.github/workflows/`. Décortiquons un exemple pour une application Node.js :

```
name: CI                                # nom du pipeline
on:                                     # quand se déclenche-t-il ?
  push:
    branches: [ main ]                 # à chaque push sur main
  pull_request:                        # et à chaque Pull Request
jobs:
  build-test:                          # un "job" = un ensemble d'étapes
    runs-on: ubuntu-latest             # sur quelle machine il tourne
    steps:
      - uses: actions/checkout@v4      # 1) récupère le code
      - uses: actions/setup-node@v4    # 2) installe Node.js
        with:
          node-version: '20'
          cache: 'npm'                 # met en cache les dépendances
      - run: npm ci                    # 3) installe les dépendances
      - run: npm run lint --if-present # 4) vérifie le style du code
      - run: npm test                  # 5) lance les tests
```

Chaque ligne sous `steps` est une étape exécutée dans l'ordre. Si l'une échoue (par exemple un test qui ne passe pas), le pipeline s'arrête et est marqué « rouge ». Voici le test minimal que ce pipeline exécute :

```
const sum = (a, b) => a + b;

test('additionne deux nombres', () => {
  expect(sum(2, 3)).toBe(5); // si ce n'est pas 5, le test échoue
});
```

11.3 Le même pipeline avec GitLab CI

L'équivalent chez GitLab se nomme `.gitlab-ci.yml` et fonctionne sur le même principe d'étapes :

```
stages: [build, test]           # les phases, dans l'ordre

build:
  stage: build
  image: node:20                 # l'image Docker dans laquelle on travaille
  script:
    - npm ci
    - npm run build --if-present

test:
  stage: test
  image: node:20
  script:
    - npm ci
    - npm test
```

Exercice — Atelier 1 : votre premier pipeline CI

1. Créez un dépôt avec une petite application et **un** test ; vérifiez d'abord que `npm test` passe en local.
2. Ajoutez le fichier de workflow ci-dessus, poussez le code, et observez l'exécution du pipeline.
3. **Cassez volontairement** le test (remplacez `toBe(5)` par `toBe(6)`) et poussez : constatez le pipeline qui devient rouge.
4. Configurez la **protection de branche** sur `main` pour interdire toute fusion si le pipeline est rouge.

Le moment où tout devient clair

L'étape 3 de l'exercice est la plus instructive : voir une fusion **bloquée** par un test qui échoue fait comprendre, mieux que n'importe quel discours, à quoi sert l'intégration continue. C'est un filet de sécurité qui empêche le code cassé d'atteindre la branche principale.

Chapitre 12 — Docker et la conteneurisation

12.1 Le problème que Docker résout

Le « ça marche sur ma machine » a une cause technique précise : une application dépend de son **environnement** (la version exacte du langage, des bibliothèques, des variables système...). Si cet environnement diffère entre la machine du développeur, le serveur de test et la production, l'application peut se comporter différemment, voire planter.

Docker résout cela en **empaquetant l'application avec tout son environnement** dans une unité standardisée appelée un **conteneur**. Ce conteneur s'exécute de façon identique partout. Le « ça marche sur ma machine » devient « ça marche partout, à l'identique ».

12.2 Conteneur ou machine virtuelle ?

On compare souvent les conteneurs aux machines virtuelles. La différence est fondamentale :



Une **machine virtuelle** embarque un **système d'exploitation complet** : elle pèse plusieurs gigaoctets et met une à deux minutes à démarrer. Un **conteneur**, lui, **partage le noyau** du système hôte et n'embarque que l'application et ses dépendances : il pèse quelques dizaines de mégaoctets et démarre en quelques **millisecondes**. C'est cette légèreté qui rend les conteneurs si pratiques.

12.3 Le vocabulaire de Docker

Terme	Définition	Analogie
Image	Un modèle figé, en lecture seule, contenant l'application et son environnement	Le moule à gâteau (ou une « classe » en programmation)
Conteneur	Une instance de cette image, en cours d'exécution	Le gâteau sorti du moule (ou un « objet »)
Volume	Un espace de stockage qui persiste même quand le conteneur est détruit	Un disque dur externe
Réseau	Un réseau virtuel qui relie les conteneurs entre eux	Un petit réseau local privé
Registry (registre)	Un dépôt où l'on stocke et partage les images	Un « app store » d'images

12.4 Les commandes essentielles

```
docker run -d -p 8080:80 --name web nginx # lance un conteneur en arrière-plan
docker ps                                # liste les conteneurs actifs
docker logs -f web                       # affiche et suit les logs
docker exec -it web sh                   # ouvre un terminal dans le conteneur
docker stop web && docker rm web         # arrête puis supprime le conteneur
docker images                             # liste les images locales
```

Une explication s'impose sur `-p 8080:80` : cela **publie** le port 80 *interne* du conteneur sur le port 8080 de votre machine. C'est ce qui vous permet d'ouvrir `http://localhost:8080` pour accéder à l'application qui, à l'intérieur, écoute sur le port 80.

12.5 Le Dockerfile : la recette de l'image

Pour créer sa propre image, on écrit un **Dockerfile**, un fichier texte qui décrit, étape par étape, comment construire l'image. Voici un exemple **commenté** pour une application Node.js, utilisant la technique du **multi-stage build** (construction en plusieurs étapes) :

```
# ----- Étape 1 : la construction -----
FROM node:20-alpine AS build           # on part d'une image Node légère, nommée "build"
WORKDIR /app                           # le dossier de travail dans le conteneur
COPY package*.json ./                 # on copie d'abord SEULEMENT les fichiers de dépendances
RUN npm ci                             # on installe toutes les dépendances (y compris de dev)
COPY . .                               # puis on copie le reste du code source
RUN npm run build                      # on compile l'application (ex. TypeScript -> dist/)

# ----- Étape 2 : l'exécution -----
FROM node:20-alpine                   # on repart d'une image propre et légère
WORKDIR /app
ENV NODE_ENV=production
COPY package*.json ./
RUN npm ci --omit=dev                 # on installe UNIQUEMENT les dépendances de production
COPY --from=build /app/dist ./dist    # on récupère le résultat compilé de l'étape 1
EXPOSE 3000                           # on documente le port utilisé
USER node                             # on n'exécute PAS en tant que root (sécurité)
CMD ["node", "dist/server.js"]        # la commande qui démarre l'application
```

Pourquoi cette construction en deux étapes ? Parce que les **outils de compilation** (compilateurs, dépendances de développement) sont volumineux et inutiles en production. L'étape 1 compile ; l'étape 2 ne récupère que le **résultat compilé**, dans une image propre. Résultat : l'image finale est beaucoup plus **petite** et plus **sûre**.

12.6 Les bonnes pratiques du Dockerfile

- Partez d'une **image de base minimale** (`-alpine` , `-slim` , ou `distroless`).
- **Ordonnez les instructions du moins changeant au plus changeant**. Docker met en cache chaque étape ; en copiant `package.json` avant le code source, on évite de réinstaller toutes les dépendances à chaque modification du code. C'est un gain de temps considérable.
- Utilisez le **multi-stage build** pour ne pas embarquer les outils de compilation.

- Ajoutez un fichier `.dockerignore` pour exclure ce qui ne doit pas entrer dans l'image (par exemple `node_modules`, `.git`).
- N'exécutez **jamais** en tant que `root` (`USER node`).
- Donnez des **tags de version explicites** à vos images plutôt que `latest`.

```
# .dockerignore
node_modules
.git
*.log
dist
```

! Ne mettez JAMAIS de secret dans une image

Les images Docker sont constituées de couches que **n'importe qui peut inspecter**. Si vous copiez un mot de passe, une clé d'API ou un token dans l'image, il est récupérable, même si vous le « supprimez » dans une étape ultérieure (il reste dans une couche). Les secrets doivent être fournis **à l'exécution**, via des variables d'environnement ou un gestionnaire de secrets.

12.7 Docker Compose : orchestrer plusieurs conteneurs

Une vraie application a rarement un seul conteneur : il y a souvent l'application **et** sa base de données, par exemple. **Docker Compose** permet de décrire **plusieurs services** dans un seul fichier et de les démarrer ensemble :

```
services:
  api:                                # premier service : notre application
    build: .                          # construite depuis le Dockerfile local
    ports:
      - "3000:3000"
    environment:
      - DATABASE_URL=postgres://app:secret@db:5432/quickbite
    depends_on:
      - db                            # démarre après la base
    networks: [app-net]

  db:                                # second service : la base PostgreSQL
    image: postgres:16-alpine
    environment:
      POSTGRES_USER: app
      POSTGRES_PASSWORD: secret
      POSTGRES_DB: quickbite
    volumes:
      - db-data:/var/lib/postgresql/data # persiste les données
    networks: [app-net]

volumes:
  db-data:                            # le volume nommé, persistant
networks:
  app-net:                            # le réseau privé qui relie api et db
```

Remarquez que l'API se connecte à la base via l'adresse `db:5432` : à l'intérieur du réseau Compose, **chaque service est accessible par son nom**. Les commandes principales :


```
docker compose up -d --build    # construit et démarre tous les services
docker compose ps               # affiche l'état des services
docker compose logs -f api      # suit les logs d'un service
docker compose down -v          # arrête tout et supprime les volumes
```

12.8 Publier une image dans un registre

Une fois l'image construite, on la **pousse** vers un registre pour la partager :

```
docker login                    # se connecter à Docker Hub
docker build -t monuser/quickbite-api:1.0.0 .    # construire avec un tag
docker push monuser/quickbite-api:1.0.0          # publier l'image
```

Exercice — Atelier 2 : conteneuriser QuickBite

1. Écrivez le **Dockerfile** (multi-stage) et le **.dockerignore** ; construisez l'image et testez l'endpoint **/health** .
2. Ajoutez une base PostgreSQL via **docker-compose.yml** et reliez l'API au service **db** .
3. Ajoutez un **volume** et vérifiez que les données survivent à un **down** puis **up** .
4. Taguez et **poussez** l'image sur un registre.

Critère de réussite : l'image finale doit faire moins de 200 Mo (grâce au multi-stage), et **docker compose up** doit démarrer l'API et la base sans erreur.

Chapitre 13 — Kubernetes, l'orchestration

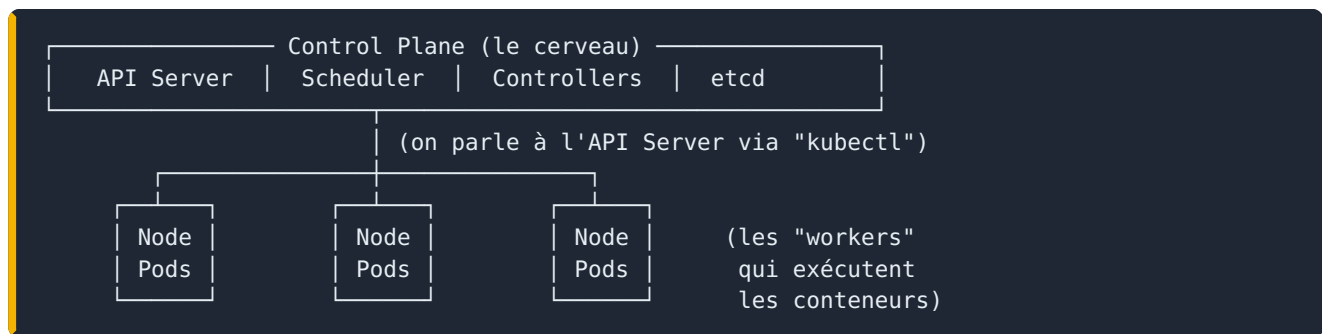
13.1 Pourquoi a-t-on besoin de Kubernetes ?

Docker lance des conteneurs **sur une seule machine**. C'est parfait en développement. Mais en **production**, on veut bien plus :

- Faire tourner l'application sur **plusieurs machines** pour résister aux pannes.
- **Redémarrer automatiquement** un conteneur qui plante.
- **Augmenter ou diminuer** le nombre de copies selon la charge (la « mise à l'échelle »).
- Déployer de nouvelles versions **sans interruption de service**.
- **Répartir la charge** entre les copies.

Faire tout cela à la main est impossible. **Kubernetes** (souvent abrégé **K8s**) est l'**orchestrateur** qui automatise tout cela. C'est devenu le standard de l'industrie.

13.2 L'architecture en deux mots



Le **Control Plane** est le cerveau qui prend les décisions ; les **Nodes** (nœuds) sont les machines qui exécutent réellement les conteneurs. On communique avec le cluster via l'outil en ligne de commande **kubectl**.

13.3 Les objets fondamentaux

Objet	Son rôle
Pod	La plus petite unité déployable : un (ou plusieurs) conteneur(s) qui partagent un réseau et un stockage
ReplicaSet	Garantit qu'un nombre <i>N</i> de copies d'un Pod tourne en permanence
Deployment	Gère les ReplicaSet : il pilote les mises à jour progressives et les retours en arrière
Service	Fournit une adresse réseau stable et répartit le trafic entre les Pods
Ingress	Gère l'accès HTTP(S) depuis l'extérieur (noms de domaine, chemins d'URL)

Une nuance importante : les Pods sont **éphémères** (ils naissent et meurent, leur adresse IP change). Le **Service** existe précisément pour fournir un point d'entrée **fixe** vers ces Pods changeants.

13.4 Un Deployment commenté

On décrit les objets dans des fichiers YAML appelés **manifestes**. Voici un Deployment :

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: quickbite-api
spec:
  replicas: 3 # je veux 3 copies de mon application
  selector:
    matchLabels: { app: quickbite-api }
  template: # le modèle de Pod à créer
    metadata:
      labels: { app: quickbite-api }
    spec:
      containers:
        - name: api
          image: ghcr.io/monuser/quickbite-api:1.0.0
          ports:
            - containerPort: 3000
          readinessProbe: # comment K8s sait que le Pod est prêt
            httpGet: { path: /health, port: 3000 }
            initialDelaySeconds: 5
          resources: # les ressources allouées
            requests: { cpu: "100m", memory: "128Mi" }
            limits: { cpu: "500m", memory: "256Mi" }
```

La `readinessProbe` est essentielle : elle dit à Kubernetes comment vérifier que le Pod est **prêt à recevoir du trafic** (ici, en appelant `/health`). Tant que la sonde échoue, Kubernetes n'envoie aucune requête à ce Pod. C'est ce qui permet les déploiements **sans coupure**.

13.5 Le Service et l'Ingress

```
apiVersion: v1
kind: Service
metadata:
  name: quickbite-api
spec:
  selector: { app: quickbite-api } # cible les Pods portant ce label
  ports:
    - port: 80
      targetPort: 3000 # redirige le port 80 vers le 3000 des Pods
  type: ClusterIP # accessible uniquement dans le cluster
```

L'**Ingress** ajoute l'accès externe par nom de domaine :

```

apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: quickbite-ingress
spec:
  rules:
    - host: quickbite.local      # le nom de domaine
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: quickbite-api
                port: { number: 80 }

```

13.6 Les commandes kubectl à connaître

```

kubectl apply -f deployment.yaml      # crée ou met à jour depuis un manifeste
kubectl get pods,svc,deploy          # liste les objets
kubectl describe pod <nom>           # détails et événements (l'outil de debug n°1)
kubectl logs -f <pod>                # affiche et suit les logs
kubectl scale deploy/quickbite-api --replicas=5 # passe à 5 copies
kubectl rollout undo deploy/quickbite-api      # revient à la version précédente

```

Le déclaratif : la grande idée de Kubernetes

On ne dit pas à Kubernetes « lance ce conteneur » (impératif). On lui décrit l'**état souhaité** : « je veux 3 copies de cette application en permanence » (déclaratif). Kubernetes compare en continu cet état souhaité à l'état réel, et **agit pour les faire correspondre**. Si un Pod meurt, il en recrée un, automatiquement, sans qu'on intervienne. C'est ce qu'on appelle l'**auto-réparation**, et c'est aussi le fondement du GitOps qu'on verra au chapitre 14.

13.7 Configuration et secrets

On ne met pas la configuration « en dur » dans l'image (sinon il faudrait reconstruire l'image à chaque changement de paramètre). On utilise des **ConfigMaps** (pour la configuration ordinaire) et des **Secrets** (pour les données sensibles) :

```

apiVersion: v1
kind: ConfigMap
metadata: { name: quickbite-config }
data:
  APP_MODE: "production"
  PAGE_SIZE: "20"
---
apiVersion: v1
kind: Secret
metadata: { name: quickbite-secret }
type: Opaque
stringData:
  DATABASE_PASSWORD: "secret-a-changer"

```

⚠ Un Secret Kubernetes est encodé, pas chiffré

Attention à un piège classique : par défaut, un Secret est seulement encodé en base64 — ce qui se décode en une seconde. Ce n'est **pas** du chiffrement. Pour une vraie protection, il faut activer le chiffrement « au repos » du stockage de Kubernetes (etcd), ou utiliser un gestionnaire externe (Vault, Sealed Secrets). Et bien sûr, ne committez **jamais** un Secret en clair dans Git.

Les **Namespaces** (espaces de noms) cloisonnent les ressources, par exemple pour séparer `dev`, `staging` et `prod` dans un même cluster.

13.8 Un mot sur la sécurité (RBAC)

Le **RBAC** (*Role-Based Access Control*, contrôle d'accès basé sur les rôles) définit **qui** a le droit de faire **quoi** dans le cluster. On crée des `Role` (un ensemble de permissions) qu'on associe à des utilisateurs via des `RoleBinding`. Le principe directeur est celui du **moindre privilège** : chacun ne reçoit que les droits strictement nécessaires à son travail.

🔧 Exercice — Atelier 3 : déployer sur un cluster local

Avec **Minikube** ou **K3s** :

1. Démarrez le cluster (`minikube start`) et activez l'Ingress.
2. Appliquez le Deployment et le Service ; vérifiez que 3 Pods sont « Running ».
3. Ajoutez une ConfigMap et un Secret, et injectez-les dans l'application.
4. Exposez l'application via l'Ingress et testez-la dans le navigateur.
5. Passez à 5 réplicas avec `kubectl scale`, puis déployez une **mauvaise** image et faites un `rollout undo` pour revenir en arrière.

Réflexe de debug à acquérir : en cas de souci, votre premier geste est toujours `kubectl describe pod <nom>` et de lire la section « Events » en bas.

Chapitre 14 — La chaîne complète : Helm, observabilité, GitOps

14.1 Assembler une chaîne CI/CD de bout en bout

L'objectif final est qu'à chaque nouvelle version (par exemple un tag Git), le pipeline **construise l'image, la pousse dans le registre, et la déploie** automatiquement sur Kubernetes :

```

name: CI-CD
on:
  push:
    tags: [ 'v*' ]           # se déclenche sur un tag commençant par v
jobs:
  build-push:
    runs-on: ubuntu-latest
    permissions: { contents: read, packages: write }
    steps:
      - uses: actions/checkout@v4
      - uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${ github.actor }
          password: ${ secrets.GITHUB_TOKEN }
      - uses: docker/build-push-action@v6
        with:
          context: .
          push: true
          tags: ghcr.io/${ github.repository }:${ github.ref_name }
  deploy:
    needs: build-push        # ne s'exécute qu'après le build
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: |
          kubectrl set image deploy/quickbite-api \
            api=ghcr.io/${ github.repository }:${ github.ref_name } \
            --namespace=prod

```

14.2 Helm : le gestionnaire de paquets de Kubernetes

Quand on a plusieurs environnements (dev, staging, prod), on se retrouve à dupliquer des manifestes YAML quasi identiques, qui ne diffèrent que par quelques valeurs. C'est lourd et source d'erreurs. **Helm** résout cela en **transformant les manifestes en modèles paramétrables** (des *templates*).

Un *chart* Helm est un dossier structuré :

```

quickbite-chart/
├─ Chart.yaml           # les métadonnées du chart
├─ values.yaml          # les valeurs par défaut (paramètres)
└─ templates/
  ├─ deployment.yaml    # un manifeste avec des {{ .Values.* }}
  ├─ service.yaml
  └─ ingress.yaml

```

Le fichier `values.yaml` centralise les paramètres :

```

replicaCount: 3
image:
  repository: ghcr.io/monuser/quickbite-api
  tag: "1.0.0"
service:
  port: 80

```

Et le template y fait référence avec une syntaxe `{{ .Values.xxx }}` :

```
spec:
  replicas: {{ .Values.replicaCount }}
  template:
    spec:
      containers:
        - name: api
          image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
```

On peut alors déployer le même chart avec des valeurs différentes par environnement :

```
helm install quickbite ./quickbite-chart -n prod
helm upgrade quickbite ./quickbite-chart --set image.tag=1.1.0 -n prod
helm rollback quickbite 1 -n prod # revenir à la version 1
```

14.3 L'observabilité : voir ce qui se passe en production

Une fois en production, comment sait-on que tout va bien ? Grâce à l'**observabilité**, qui repose sur trois piliers :

- Les **logs** (journaux) : *que s'est-il passé ?*
- Les **métriques** : *combien, et à quelle vitesse ?* (nombre de requêtes, temps de réponse...)
- Les **traces** : *quel chemin une requête a-t-elle suivi à travers les différents services ?*

Le duo le plus répandu pour les métriques est **Prometheus** (qui collecte et stocke les métriques) et **Grafana** (qui les affiche sous forme de tableaux de bord) :

```
Application (/metrics) —collecte—> Prometheus —affichage—> Grafana (dashboards)
                                   |
                                   +--> Alertmanager —> email / Slack
```

Les quatre signaux dorés

Par où commencer quand on supervise un service ? Les ingénieurs de fiabilité de Google recommandent de surveiller en priorité quatre indicateurs, les « **signaux dorés** » : la **latence** (temps de réponse), le **trafic** (nombre de requêtes), les **erreurs** (taux d'échec) et la **saturation** (à quel point les ressources sont remplies). Un tableau de bord qui couvre ces quatre signaux détecte la grande majorité des problèmes.

14.4 Le GitOps avec Argo CD

Le **GitOps** est l'aboutissement de la logique déclarative de Kubernetes. L'idée : l'**état souhaité du système vit entièrement dans Git**, qui devient la **source unique de vérité**. Un agent installé dans le cluster — par exemple **Argo CD** — compare en permanence ce qui est décrit dans Git avec ce qui tourne réellement, et **synchronise** automatiquement.

```
Développeur → commit / Pull Request → Dépôt Git (manifestes, charts Helm)
|
Argo CD surveille en continu
▼
Cluster Kubernetes (se met à jour pour
correspondre à ce qui est décrit dans Git)
```

Les avantages sont considérables : tout changement passe par Git, donc tout est **tracé et auditable** ; un retour en arrière se fait par un simple `git revert` ; et il n'y a plus de `kubectl apply` manuel et risqué en production. **Git devient le tableau de bord et le bouton de déploiement.**

Exercice — Atelier final : du commit à la production

Assemblez toute la chaîne du module : (1) un pipeline qui **teste** puis **construit et pousse** l'image ; (2) un **chart Helm** paramétré ; (3) un **déploiement** sur le cluster local exposé par Ingress ; (4) un tableau de bord **Grafana** affichant au moins un signal doré ; (5) en bonus, l'application placée sous **Argo CD** pour qu'un commit déclenche le déploiement.

Démonstration finale : racontez, en partant d'une modification de code, tout le trajet « du commit à la production ». C'est la synthèse de tout le module.

14.5 Récapitulatif de la partie conceptuelle

Retenez la chaîne logique : **Git** stocke le code, la **CI** le teste à chaque changement, **Docker** l'empaquette de façon portable, un **registre** le distribue, **Kubernetes** l'exécute de façon résiliente et scalable, **Helm** paramètre les déploiements, **l'observabilité** surveille le tout, et le **GitOps** boucle la boucle en pilotant la production depuis Git. Le but de toute cette mécanique n'est pas la technique pour la technique : c'est de **livrer la valeur agile du Module 1 plus vite et plus sûrement**.

La théorie ne suffit pas : un outil DevOps ne s'apprend qu'**en l'utilisant**. La partie suivante est un **cours pratique pas à pas** où vous tapez réellement chaque commande. On part du système d'exploitation (Linux), on conteneurise l'application (Docker), on l'orchestre (Kubernetes), on collabore proprement (GitHub) et on automatise tout (GitHub Actions). Le fil rouge reste **QuickBite**, qui est une vraie application **Spring Boot (Java 21, Maven)** exposée sur le port **8080** avec un endpoint de santé `/health`.

MODULE 2 · PARTIE PRATIQUE — Linux, Docker, Kubernetes, GitHub & CI/CD

■ De quoi avez-vous besoin pour suivre

Un environnement **Linux** : soit une machine ou une VM Ubuntu, soit **WSL2** sous Windows (`wsl --install`), soit un Mac (les commandes shell sont quasi identiques). Vous installerez au fil de l'eau : **Docker**, **kubectl** + un cluster local (**minikube** ou **kind**), **git**, le client **GitHub CLI** (`gh`) et un **JDK 21** + **Maven** pour QuickBite. Chaque atelier indique sa commande d'installation. Tapez les commandes vous-même : la mémoire des mains compte autant que celle de la tête.

Atelier Linux — le système, le terminal et le shell

Tout, en DevOps, finit sur **Linux** : vos conteneurs Docker tournent sur un noyau Linux, vos nœuds Kubernetes sont des machines Linux, les *runners* GitHub Actions par défaut sont des `ubuntu-latest`. Savoir se déplacer dans un terminal Linux n'est donc pas optionnel — c'est le socle de tout le reste.

L.1 Le shell, c'est quoi ?

Quand vous ouvrez un terminal, vous parlez à un programme appelé le **shell** (le plus courant est **bash**, un autre très répandu est **zsh**). Il lit ce que vous tapez, l'exécute, et vous rend la main. L'invite (le *prompt*) ressemble souvent à ceci :

```
haithem@laptop:~/projets/quickbite$  
|           |           |           |  
utilisateur  machine  dossier courant  $ = utilisateur normal (# = root)
```

Le caractère `~` est un raccourci pour **votre dossier personnel** (`/home/haithem`). Le `$` final indique que vous êtes un utilisateur normal ; un `#` signifierait que vous êtes `root` (l'administrateur tout-puissant).

L.2 L'arborescence des fichiers

Sous Linux, **tout est un fichier** (même un périphérique ou un processus), et tout part d'une racine unique, `/`. Les dossiers que vous croiserez le plus :

Chemin	Contenu
/	la racine de tout
/home/<user>	vos fichiers personnels
/etc	les fichiers de configuration du système
/var	les données variables : logs (/var/log), bases, files d'attente
/tmp	les fichiers temporaires (effacés au redémarrage)
/usr/bin , /bin	les programmes installés
/opt	les logiciels tiers installés à la main
/proc , /sys	des vues virtuelles du noyau et du matériel

Un **chemin absolu** commence par / (/etc/hosts) ; un **chemin relatif** part du dossier courant (src/main). Deux raccourcis essentiels : . = le dossier courant, .. = le dossier parent.

L.3 Se déplacer et manipuler des fichiers

```

pwd                # "print working directory" : où suis-je ?
ls                 # lister le contenu du dossier
ls -la            # -l = détaillé, -a = inclut les fichiers cachés (.xxx)
cd /etc           # aller dans /etc
cd ..             # remonter d'un cran
cd                # revenir dans son dossier personnel (~)
cd -              # revenir au dossier précédent

mkdir -p projets/quickbite # créer une arborescence (-p crée les parents)
touch notes.txt           # créer un fichier vide (ou actualiser sa date)
cp notes.txt sauvegarde.txt # copier
cp -r dossier/ copie/      # copier un dossier entier (-r = récursif)
mv notes.txt docs/         # déplacer (ou renommer)
rm sauvegarde.txt          # supprimer un fichier
rm -r copie/              # supprimer un dossier et son contenu

```

! rm -rf ne pardonne pas

Il n'y a **pas de corbeille** en ligne de commande. `rm -rf /chemin` efface définitivement et sans confirmation. Une faute de frappe comme un espace de trop (`rm -rf / tmp` au lieu de `rm -rf /tmp`) peut détruire le système. Relisez toujours une commande `rm -rf` **avant** d'appuyer sur Entrée, et préférez des chemins absolus explicites.

L.4 Lire le contenu des fichiers

```
cat application.yml           # afficher tout le fichier d'un coup
less /var/log/syslog         # parcourir un gros fichier (q pour quitter, / pour chercher)
head -n 20 fichier.log       # les 20 premières lignes
tail -n 50 fichier.log       # les 50 dernières lignes
tail -f /var/log/app.log     # SUIVRE en direct (idéal pour observer des logs)
wc -l fichier.log            # compter les lignes
```

`tail -f` (« follow ») est un réflexe à acquérir : il affiche les nouvelles lignes au fur et à mesure qu'elles sont écrites. C'est exactement ce que fait `docker logs -f` ou `kubectl logs -f`.

L.5 Chercher : `grep`, `find`

```
grep "ERROR" app.log         # toutes les lignes contenant ERROR
grep -i "error" app.log      # -i = insensible à la casse
grep -r "TODO" src/         # -r = chercher récursivement dans une arbo
grep -n "port" application.yml # -n = afficher les numéros de ligne
grep -c "200" access.log     # -c = compter les occurrences

find . -name "*.java"        # tous les fichiers .java sous le dossier courant
find . -type d -name target   # tous les DOSSIERS (-type d) nommés target
find /var/log -mtime -1      # fichiers modifiés il y a moins d'un jour
```

L.6 Les tuyaux (`|`) et les redirections — la vraie puissance d'Unix

La philosophie Unix : de **petits outils** qui font **une seule chose**, qu'on **enchaîne**. Le « tuyau » `|` envoie la sortie d'une commande comme entrée de la suivante.

```
cat access.log | grep "500" | wc -l    # combien d'erreurs 500 dans le log ?
ps aux | grep java                    # tous les processus liés à java
ls -la | sort -k5 -n                  # trier les fichiers par taille

commande > sortie.txt                 # rediriger la sortie vers un fichier (écrase)
commande >> sortie.txt                 # ajouter à la fin du fichier (n'écasse pas)
commande 2> erreurs.txt               # rediriger les ERREURS (flux 2) vers un fichier
commande > tout.txt 2>&1               # rediriger sortie ET erreurs au même endroit
commande < entree.txt                 # lire l'entrée depuis un fichier
```

Et les enchaînements conditionnels, omniprésents dans les scripts CI/CD :

```
mvn test && docker build -t app .    # build SEULEMENT si les tests passent (&&)
ping -c1 serveur || echo "injoignable" # message SI la commande échoue (||)
```



```
ps aux                # lister TOUS les processus (a=tous, u=détaillé, x=arrière-plan)
ps aux | grep java    # ne garder que ceux liés à java
top                   # moniteur temps réel (CPU, mémoire) – q pour quitter
htop                  # version plus lisible (à installer : sudo apt install htop)

kill 1234              # demander poliment au processus 1234 de s'arrêter (SIGTERM)
kill -9 1234           # le forcer brutalement (SIGKILL) – en dernier recours
pkill -f quickbite     # tuer par nom/motif de commande
```

Lancer un programme en **arrière-plan** et le garder en vie :

```
./serveur.sh &        # & = lancer en arrière-plan
nohup ./serveur.sh &  # nohup = survit à la fermeture du terminal
jobs                  # voir les tâches d'arrière-plan du shell
```

La distinction **SIGTERM** (15, « range tes affaires et arrête-toi ») vs **SIGKILL** (9, « arrêt immédiat sans préavis ») est importante : c'est exactement ce que fait Docker quand il arrête un conteneur (**stop** envoie SIGTERM puis SIGKILL après un délai de grâce).

L.10 Installer des logiciels (gestionnaire de paquets)

Sur les distributions Debian/Ubuntu, le gestionnaire est **apt** :

```
sudo apt update        # rafraîchir la liste des paquets disponibles
sudo apt install -y htop git curl # installer des paquets (-y = sans confirmation)
sudo apt upgrade       # mettre à jour les paquets installés
sudo apt remove htop   # désinstaller
apt search docker      # chercher un paquet
```

Sur Red Hat/Fedora/CentOS, c'est **dnf** (ou l'ancien **yum**) avec la même logique : **sudo dnf install ...**. **À retenir** : sur Debian/Ubuntu, **apt update** ne **met pas à jour** les logiciels — il met à jour la **liste** ; c'est **apt upgrade** qui installe les nouvelles versions.

L.11 Les services et **systemd**

Un **service** (ou *daemon*) est un programme qui tourne en permanence en arrière-plan (un serveur web, une base de données...). Sur Linux moderne, c'est **systemd** qui les gère, via la commande **systemctl** :

```
sudo systemctl status docker # l'état d'un service (actif ? depuis quand ?)
sudo systemctl start docker  # démarrer
sudo systemctl stop docker   # arrêter
sudo systemctl restart docker # redémarrer
sudo systemctl enable docker # le lancer AUTOMATIQUEMENT au démarrage de la machine
sudo systemctl disable docker # ne plus le lancer au démarrage

journalctl -u docker          # les logs du service docker
journalctl -u docker -f       # les suivre en direct
journalctl -u docker --since "10 min ago"
```

La nuance `start` vs `enable` revient souvent : `start` lance **maintenant** ; `enable` fait en sorte qu'il se relance **tout seul après un redémarrage**. On combine généralement les deux : `sudo systemctl enable --now docker`.

L.12 Le réseau

```
ip a          # afficher les interfaces réseau et leurs adresses IP
ss -tulpn     # les ports en écoute (t=tcp, u=udp, l=listen, p=process, n=numérique)
ping -c4 google.com # tester la connectivité (4 paquets)
curl -i http://localhost:8080/health # appeler une URL HTTP et voir les en-têtes (-i)
curl -s http://localhost:8080/actuator/health | jq # -s silencieux, jq formate le JSON
```

Pour se connecter à une machine distante et y copier des fichiers :

```
ssh haithem@192.168.1.10 # ouvrir une session distante
scp app.jar haithem@serveur:/opt/ # copier un fichier vers la machine distante
```

`ss -tulpn` est l'outil n°1 quand un service « ne répond pas » : il dit **quel programme écoute sur quel port**. Si votre `:8080` n'apparaît pas, l'application n'est pas démarrée ou écoute ailleurs.

L.13 Variables d'environnement

Une **variable d'environnement** est une valeur nommée que le système transmet aux programmes. C'est le moyen standard de configurer une application **sans toucher à son code** (et c'est central en Docker/Kubernetes) :

```
echo $HOME          # afficher une variable existante
echo $PATH           # la liste des dossiers où le shell cherche les programmes
export SPRING_PROFILES_ACTIVE=postgres # définir une variable pour la session
export DATABASE_URL="jdbc:postgresql://db:5432/quickbite"
printenv             # lister toutes les variables d'environnement
```

Le `PATH` mérite une explication : quand vous tapez `docker`, le shell cherche un programme nommé `docker` dans chacun des dossiers listés dans `$PATH`, dans l'ordre. C'est pourquoi un outil « introuvable » l'est souvent simplement parce que son dossier n'est pas dans le `PATH`. Pour rendre une variable permanente, on l'ajoute au fichier `~/.bashrc` (ou `~/.zshrc`), lu à chaque ouverture de terminal.

L.14 Écrire un script shell

Un **script** est une suite de commandes dans un fichier, qu'on exécute d'un coup. C'est la base de l'automatisation (et une étape de pipeline CI/CD n'est souvent rien d'autre qu'un script). Créez `deploy.sh` :

```
#!/usr/bin/env bash
# La première ligne (le "shebang") indique avec quel interpréteur exécuter le fichier.

set -euo pipefail
# -e : arrêter au premier échec. -u : erreur si une variable est indéfinie.
# -o pipefail : un échec au milieu d'un | fait échouer toute la chaîne.

APP_NAME="quickbite-api"           # une variable
VERSION="${1:-latest}"             # $1 = 1er argument ; "latest" par défaut

echo "► Construction de ${APP_NAME}:${VERSION}"

if [ -f "pom.xml" ]; then           # tester l'existence d'un fichier
    mvn -B clean package -DskipTests
else
    echo "Erreur : pom.xml introuvable" >&2 # écrire sur la sortie d'erreur
    exit 1                                # quitter avec un code d'échec
fi

for env in dev staging prod; do     # une boucle
    echo " → préparation de l'environnement ${env}"
done

echo "✅ Terminé."
```

On le rend exécutable puis on le lance :

```
chmod +x deploy.sh
./deploy.sh 1.2.0                 # "1.2.0" devient $1
echo $?                           # afficher le CODE DE SORTIE de la dernière commande (0 = succès)
```

Le **code de sortie** (`exit code`) est fondamental : par convention, **0 = succès**, tout autre nombre = échec. C'est sur ce code que reposent `&&`, `||` et **toute** la logique des pipelines CI : une étape « rouge », c'est simplement une commande qui a renvoyé un code différent de 0.

Exercice — Atelier Linux

Sur votre machine Linux/WSL :

1. Créez l'arborescence `~/labs/quickbite/{src,logs,config}` en **une** commande `mkdir -p`.
2. Générez un faux log : `for i in $(seq 1 100); do echo "$i $((RANDOM%500+100)) /api/orders"; done > logs/access.log`.
3. Avec un **tuyau**, comptez combien de lignes contiennent un code `500`, puis affichez le **top 3** des codes les plus fréquents (`awk + sort + uniq -c`).
4. Écrivez un script `healthcheck.sh` qui appelle `curl -s http://localhost:8080/health` et affiche « OK » ou « KO » selon le **code de sortie** de `curl`, en utilisant `set -euo pipefail`.
5. Rendez-le exécutable et vérifiez `echo $?`.

Objectif : être à l'aise pour naviguer, filtrer des logs et écrire un script simple — ce sont les gestes que vous referez dans chaque atelier suivant.

Atelier Docker pratique — de l'image au conteneur

Le chapitre 12 a expliqué **pourquoi** Docker. Place à la pratique : on installe, on lance, on construit, on débogue, et on conteneurise réellement QuickBite.

D.1 Installer et vérifier

```
# Ubuntu : script officiel (pour un poste de dev)
curl -fsSL https://get.docker.com | sudo sh
sudo usermod -aG docker $USER          # pour utiliser docker sans sudo (se reconnecter ensuite)

docker version                         # versions client + serveur
docker run hello-world                 # le "hello world" : télécharge et lance une image test
docker info                           # état du démon, nombre d'images/conteneurs
```

Si `hello-world` affiche son message, votre installation fonctionne : Docker a **téléchargé** une image depuis Docker Hub, **créé** un conteneur, l'a **exécuté**, puis il s'est arrêté.

D.2 Le cycle de vie d'un conteneur

```
docker run -d -p 8080:80 --name web nginx # -d détaché, -p publie le port, --name le nomme
docker ps                                # les conteneurs EN COURS
docker ps -a                             # TOUS, y compris arrêtés
docker logs web                           # les logs du conteneur
docker logs -f web                        # les suivre en direct
docker exec -it web bash                  # ouvrir un shell DANS le conteneur (-it interactif)
docker stop web                           # arrêt propre (SIGTERM puis SIGKILL)
docker start web                           # le relancer
docker rm -f web                          # le supprimer (-f force même s'il tourne)
```

Ouvrez `http://localhost:8080` : la page d'accueil Nginx s'affiche, servie depuis le conteneur. La commande `docker exec -it web bash` est votre meilleur ami pour **inspecter de l'intérieur** un conteneur qui se comporte mal.

D.3 Images : lister, inspecter, nettoyer

```
docker images                         # les images locales
docker pull postgres:16-alpine        # télécharger une image sans la lancer
docker inspect nginx                   # toutes les métadonnées (JSON)
docker history monimage:1.0            # voir les COUCHES qui composent l'image
docker rmi nginx                       # supprimer une image

docker system df                       # combien d'espace Docker occupe
docker system prune -a                 # GRAND nettoyage : images/conteneurs/réseaux inutilisés
```


💡 Les couches et le cache, vus en vrai

Lancez `docker history` sur une de vos images : chaque ligne est une **couche**, créée par une instruction du Dockerfile. Docker **réutilise** une couche tant que ce qui la précède n'a pas changé. C'est la raison pour laquelle on copie `pom.xml` (ou `package.json`) **avant** le code source : tant que les dépendances ne bougent pas, la couche `mvn dependency:go-offline` reste en cache et le rebuild prend quelques secondes au lieu de plusieurs minutes.

D.4 Construire l'image de QuickBite (Spring Boot, multi-stage)

QuickBite est une application **Spring Boot / Maven / Java 21**. Voici le `Dockerfile` multi-stage, commenté — c'est exactement le motif utilisé en production pour une appli Java :

```
# ----- Étape 1 : build (avec Maven + JDK) -----
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /app
COPY pom.xml .
RUN mvn -B dependency:go-offline      # couche STABLE : ne se reconstruit que si pom.xml change
COPY src ./src
RUN mvn -B clean package -DskipTests # produit target/quickbite-api-1.0.0.jar

# ----- Étape 2 : runtime (JRE seule, image légère) -----
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
ENV SPRING_PROFILES_ACTIVE=postgres
COPY --from=build /app/target/quickbite-api-*.jar app.jar # on ne récupère QUE le .jar
EXPOSE 8080
RUN addgroup -S app && adduser -S app -G app # créer un utilisateur non-root
USER app # ne jamais tourner en root
ENTRYPOINT ["java", "-jar", "app.jar"]
```

On construit puis on lance :

```
docker build -t quickbite-api:1.0.0 . # construire l'image (le . = contexte)
docker images | grep quickbite        # vérifier sa taille (~200 Mo grâce à la JRE alpine)
docker run -d -p 8080:8080 --name qb quickbite-api:1.0.0
curl http://localhost:8080/health     # tester l'endpoint de santé
docker logs -f qb                     # observer le démarrage de Spring
```

L'image finale n'embarque **ni Maven ni le JDK complet** : seulement une **JRE** + le `.jar`. C'est tout l'intérêt du multi-stage : une image plus **petite**, plus **rapide** à télécharger, et avec **moins de surface d'attaque**.

D.5 Variables d'environnement et configuration

On ne reconstruit pas une image pour changer un paramètre : on injecte la configuration **au lancement**. Spring lit naturellement les variables d'environnement :

```
docker run -d -p 8080:8080 \
  -e SPRING_PROFILES_ACTIVE=postgres \
  -e SPRING_DATASOURCE_URL="jdbc:postgresql://db:5432/quickbite" \
  -e SPRING_DATASOURCE_PASSWORD="secret" \
  --name qb quickbite-api:1.0.0
```

! Le mot de passe en clair dans l'historique

Passer un secret avec `-e MOT_DE_PASSE=...` le rend visible dans `docker inspect` et dans l'historique de votre shell (`history`). En production, on utilise un fichier `--env-file` non versionné, ou un vrai gestionnaire de secrets (Docker secrets, Vault, secrets Kubernetes). On ne met **jamais** un secret dans le Dockerfile ni dans l'image.

D.6 Volumes et persistance

Un conteneur est **éphémère** : quand on le supprime, ses données disparaissent. Pour persister (une base de données, par exemple), on utilise un **volume** :

```
docker volume create qb-data # créer un volume nommé
docker run -d --name db \
  -v qb-data:/var/lib/postgresql/data \ # monter le volume dans le conteneur
  -e POSTGRES_PASSWORD=secret postgres:16-alpine

docker volume ls # lister les volumes
docker run --rm -v $(pwd):/app -w /app maven:3.9-eclipse-temurin-21 mvn test
#      ^ bind mount : monter le DOSSIER COURANT de l'hôte dans le conteneur
```

Deux mécanismes à distinguer : le **volume nommé** (géré par Docker, idéal pour les données) et le **bind mount** (`-v /chemin/hote:/chemin/conteneur` , idéal en développement pour voir ses modifications de code en direct).

D.7 Réseaux entre conteneurs

```
docker network create qb-net # créer un réseau privé
docker run -d --name db --network qb-net postgres:16-alpine
docker run -d --name qb --network qb-net -p 8080:8080 quickbite-api:1.0.0
```

Sur un même réseau Docker, **un conteneur en joint un autre par son nom** : l'API atteint la base via l'hôte `db` (et non `localhost`). C'est ce qui permet à QuickBite d'écrire `jdbc:postgresql://db:5432/...`.

D.8 Docker Compose : tout démarrer ensemble

Plutôt que d'enchaîner les `docker run` , on décrit l'ensemble dans un `docker-compose.yml` . Voici la stack QuickBite (API + base), avec un **healthcheck** :

```
services:
  api:
    build: . # construit depuis le Dockerfile local
    ports: ["8080:8080"]
    environment:
      SPRING_PROFILES_ACTIVE: postgres
      SPRING_DATASOURCE_URL: jdbc:postgresql://db:5432/quickbite
      SPRING_DATASOURCE_USERNAME: app
      SPRING_DATASOURCE_PASSWORD: secret
    depends_on:
      db:
        condition: service_healthy # attend que la base soit VRAIMENT prête
    healthcheck:
      test: ["CMD", "wget", "-q0-", "http://localhost:8080/health"]
      interval: 10s
      timeout: 3s
      retries: 5

  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_USER: app
      POSTGRES_PASSWORD: secret
      POSTGRES_DB: quickbite
    volumes: ["qb-data:/var/lib/postgresql/data"]
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U app"]
      interval: 5s
      retries: 5

volumes:
  qb-data:
```

```
docker compose up -d --build # construit et démarre tout
docker compose ps            # l'état (et la santé) de chaque service
docker compose logs -f api   # suivre les logs de l'API
docker compose down          # tout arrêter (ajouter -v pour effacer les volumes)
```

Le `depends_on ... condition: service_healthy` règle un piège classique : sans lui, l'API démarre avant que PostgreSQL n'accepte les connexions et plante au lancement. Le **healthcheck** est la bonne façon de gérer l'ordre de démarrage réel.

Exercice — Atelier Docker

Dans le dossier du projet `atelier-quickbite` :

1. Construisez l'image avec `docker build -t quickbite-api:1.0.0 .` et notez sa **taille** (`docker images`).
2. Lancez-la seule (profil H2 en mémoire) et vérifiez `curl localhost:8080/health`.
3. Lancez la stack complète avec `docker compose up -d --build` ; vérifiez que `db` est `healthy` avant que `api` ne démarre.
4. Ouvrez un shell dans le conteneur API (`docker exec -it <id> sh`) et confirmez qu'il tourne en utilisateur **non-root** (commande `whoami`).
5. Faites `docker compose down`, relancez, et vérifiez que les données ont **survécu** grâce au volume.

Critère de réussite : image < 250 Mo, conteneur non-root, et persistance des données confirmée.

Atelier Kubernetes pratique — déployer pour de vrai

Le chapitre 13 a posé les concepts. Ici on **monte un cluster local** et on y déploie QuickBite, on l'expose, on le met à l'échelle, on le met à jour et on le débogue.

K.1 Monter un cluster local

Plusieurs options, toutes gratuites et légères :

```
# Option A – minikube (très répandu, démarre une "machine" k8s locale)
curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64
sudo install minikube-linux-amd64 /usr/local/bin/minikube
minikube start --driver=docker

# Option B – kind (Kubernetes IN Docker, idéal pour la CI)
# Option C – k3d (k3s léger dans Docker)

# Installer kubectl (le client en ligne de commande)
curl -LO "https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
sudo install kubectl /usr/local/bin/kubectl

kubectl version --client
kubectl get nodes # le(s) nœud(s) de votre cluster
```

K.2 Charger l'image dans le cluster

Sur un cluster local, le cluster ne voit pas automatiquement les images de votre Docker local. On les y charge :

```
minikube image load quickbite-api:1.0.0 # avec minikube
# ou, pour kind : kind load docker-image quickbite-api:1.0.0
```

K.3 Premiers objets : Deployment + Service

Créez `deployment.yaml` :

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: quickbite-api
spec:
  replicas: 2                                # 2 copies pour la résilience
  selector:
    matchLabels: { app: quickbite-api }
  template:
    metadata:
      labels: { app: quickbite-api }
    spec:
      containers:
        - name: api
          image: quickbite-api:1.0.0
          imagePullPolicy: IfNotPresent      # utiliser l'image locale chargée
          ports: [{ containerPort: 8080 }]
          env:
            - name: SPRING_PROFILES_ACTIVE
              value: "default"                # H2 en mémoire pour l'atelier
          readinessProbe:                     # PRÊT à recevoir du trafic ?
            httpGet: { path: /actuator/health, port: 8080 }
            initialDelaySeconds: 10
          livenessProbe:                      # toujours VIVANT ? (sinon redémarrage)
            httpGet: { path: /actuator/health, port: 8080 }
            initialDelaySeconds: 20
          resources:
            requests: { cpu: "200m", memory: "256Mi" }
            limits:   { cpu: "1",    memory: "512Mi" }
---
apiVersion: v1
kind: Service
metadata:
  name: quickbite-api
spec:
  selector: { app: quickbite-api }
  ports: [{ port: 80, targetPort: 8080 }]
  type: ClusterIP
```

```
kubectl apply -f deployment.yaml    # créer/mettre à jour les objets
kubectl get pods -w                 # observer les Pods passer à "Running" (-w = watch)
kubectl get deploy,svc,pods         # vue d'ensemble
```

K.4 Accéder à l'application

```
kubectl port-forward svc/quickbite-api 8080:80 # rediriger un port local vers le Service
# dans un autre terminal :
curl http://localhost:8080/health
```

`port-forward` est la façon la plus simple de tester un Service interne depuis votre machine, sans Ingress.

K.5 Configuration et secrets

```
apiVersion: v1
kind: ConfigMap
metadata: { name: quickbite-config }
data:
  APP_PAGE_SIZE: "20"
---
apiVersion: v1
kind: Secret
metadata: { name: quickbite-secret }
type: Opaque
stringData:
  SPRING_DATASOURCE_PASSWORD: "secret-a-changer"
```

On les injecte dans le conteneur via `envFrom` :

```
containers:
- name: api
  image: quickbite-api:1.0.0
  envFrom:
    - configMapRef: { name: quickbite-config }
    - secretRef: { name: quickbite-secret }
```

Rappel du chapitre 13 : un `Secret` est **encodé** en base64, **pas chiffré**. Ne le committez jamais en clair dans Git.

K.6 Sondes, mise à jour progressive et rollback

C'est là que Kubernetes brille. Changez l'image vers une nouvelle version et observez le **rolling update** (mise à jour sans coupure, Pod par Pod) :

```
kubectl set image deploy/quickbite-api api=quickbite-api:1.1.0
kubectl rollout status deploy/quickbite-api # suivre la progression du déploiement
kubectl rollout history deploy/quickbite-api # l'historique des versions
kubectl rollout undo deploy/quickbite-api # REVENIR à la version précédente
```

Grâce à la `readinessProbe`, Kubernetes n'envoie du trafic à un nouveau Pod **que** lorsqu'il répond sur `/actuator/health`. Si la nouvelle version est cassée, les anciens Pods restent en service : **pas de coupure**, et un `rollout undo` annule tout en quelques secondes.

K.7 Mise à l'échelle

```
kubectl scale deploy/quickbite-api --replicas=5 # passer à 5 copies manuellement
kubectl get pods # voir les nouveaux Pods apparaître

# Mise à l'échelle AUTOMATIQUE selon le CPU (HPA) :
kubectl autoscale deploy/quickbite-api --min=2 --max=10 --cpu-percent=70
kubectl get hpa
```

L'**HPA** (*Horizontal Pod Autoscaler*) ajoute ou retire des Pods automatiquement quand la charge CPU dépasse/descend sous le seuil — c'est l'élasticité dont on rêvait au chapitre 13.

K.8 Débuguer dans Kubernetes

```
kubectl describe pod <nom>          # ★ l'OUTIL N°1 : voir les "Events" en bas
kubectl logs <pod>                   # les logs applicatifs
kubectl logs -f <pod>                # les suivre en direct
kubectl logs <pod> --previous        # les logs du conteneur AVANT son dernier crash
kubectl exec -it <pod> -- sh          # un shell dans le Pod
kubectl get events --sort-by=.lastTimestamp # tous les événements récents du cluster
```

⚠ Décrypter les statuts d'un Pod

Un Pod qui ne démarre pas affiche un statut parlant. **ImagePullBackOff** : l'image est introuvable (mauvais nom, ou pas chargée dans le cluster — voir K.2). **CrashLoopBackOff** : le conteneur démarre puis plante en boucle (lisez `kubectl logs --previous`). **Pending** : aucun nœud ne peut l'accueillir (ressources insuffisantes). Le réflexe est **toujours** le même : `kubectl describe pod <nom>`, puis lire la section « Events ».

K.9 Exposer via un Ingress

```
minikube addons enable ingress      # activer le contrôleur Ingress
```

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: quickbite-ingress
spec:
  rules:
    - host: quickbite.local
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: quickbite-api
                port: { number: 80 }
```

```
kubectl apply -f ingress.yaml
echo "$(minikube ip) quickbite.local" | sudo tee -a /etc/hosts # résolution du nom
curl http://quickbite.local/health
```

Exercice — Atelier Kubernetes

1. Démarrez minikube et chargez votre image `quickbite-api:1.0.0`.
2. Appliquez le Deployment + Service ; vérifiez que **2 Pods** sont `Running` et que les sondes passent.
3. Accédez à `/health` via `kubectl port-forward`.
4. Provoquez une panne : `kubectl delete pod <nom>` et observez Kubernetes en **recréer un** automatiquement (auto-réparation).
5. Faites un `set image` vers une image inexistante, constatez le **ImagePullBackOff**, diagnostiquez avec `describe`, puis `rollout undo`.
6. Passez à 5 réplicas, puis exposez l'app via Ingress.

Réflexe à ancrer : en cas de souci, votre premier geste est toujours `kubectl describe pod` + lecture des Events.

Atelier GitHub — la plateforme de collaboration

Git gère les versions ; **GitHub** est la plateforme qui transforme Git en outil d'équipe : hébergement, revues de code, suivi des tâches, automatisation. C'est aussi là que vivra notre CI/CD (atelier suivant).

G.1 Créer et alimenter un dépôt

```
gh auth login                                # s'authentifier avec le GitHub CLI
gh repo create quickbite --public --source=. --remote=origin --push
# ou à la main :
git init
git add .
git commit -m "feat: initial commit du projet QuickBite"
git branch -M main
git remote add origin https://github.com/<vous>/quickbite.git
git push -u origin main
```

Trois fichiers à ne jamais oublier à la racine : un `README.md` (présentation, comment lancer le projet), un `.gitignore` (ce qu'on ne versionne **pas** : `target/`, `.env`, `*.class` ...), et une `LICENSE`.

`target/` et secrets : hors de Git

Pour QuickBite (Maven), `target/` contient des fichiers compilés **régénérables** — il ne doit **jamais** être versionné. Idem pour tout fichier de secrets (`.env`, clés). Si un secret a déjà été poussé, le retirer du dernier commit **ne suffit pas** : il reste dans l'historique. Il faut le **révoquer** et le considérer comme compromis.

G.2 Le flux de travail par branches et Pull Requests

Le cœur de la collaboration sur GitHub (le **GitHub Flow**, vu au chapitre 10) :


```
git switch -c feature/paiement      # 1. créer une branche dédiée à la fonctionnalité
# ... on code, on commit ...
git push -u origin feature/paiement # 2. publier la branche
gh pr create --fill                 # 3. ouvrir une Pull Request
# 4. un collègue relit, commente, approuve
gh pr merge --squash --delete-branch # 5. fusionner puis supprimer la branche
```

Une **Pull Request (PR)** n'est pas qu'un bouton « fusionner » : c'est l'**espace de revue**. On y discute le code ligne par ligne, on y voit le résultat des tests automatiques (la CI), et on n'autorise la fusion que lorsque tout est vert et approuvé.

G.3 Issues, labels, milestones, Projects

GitHub n'est pas qu'un dépôt de code, c'est aussi un outil de **gestion de projet** — qui rejoint directement le Module 1 :

- Les **Issues** : bugs, tâches, User Stories. On les **assigne**, on les **étiquette** (`bug` , `enhancement` , `priority:high`).
- Les **Milestones** : regrouper des issues vers un objectif (par ex. un sprint, une version).
- Les **Projects** : des **tableaux Kanban** (les colonnes « À faire / En cours / Terminé » du chapitre 7) directement reliés aux issues et PR.
- Un commit ou une PR qui mentionne `Closes #42` **ferme automatiquement** l'issue 42 à la fusion.

G.4 Protéger la branche `main`

C'est le réglage qui rend la CI **vraiment** utile. Dans `Settings → Branches → Add rule` sur `main` :

- ☒ **Require a pull request before merging** — interdire les push directs sur `main`.
- ☒ **Require approvals** (au moins 1 relecteur).
- ☒ **Require status checks to pass** — **interdire la fusion si la CI est rouge**.
- ☒ **Require branches to be up to date** — la branche doit être à jour avant fusion.

Le fichier `CODEOWNERS` désigne des relecteurs obligatoires par zone du code :

```
# .github/CODEOWNERS
*.java          @equipe-backend
/.github/        @equipe-devops
/k8s/           @equipe-devops
```

G.5 Releases, tags et secrets du dépôt

```
git tag -a v1.0.0 -m "Première version stable"
git push origin v1.0.0
gh release create v1.0.0 --generate-notes # génère les notes de version depuis les PR
```

Deux réglages dans `Settings` qui serviront à la CI/CD :

- **Secrets and variables → Actions** : on y stocke les valeurs sensibles (tokens, mots de passe) que les workflows liront **sans jamais les exposer** dans le code.
- **Environments** (`staging` , `production`) : on peut exiger une **approbation manuelle** avant qu'un déploiement vers `production` ne s'exécute.

Exercice — Atelier GitHub

1. Poussez QuickBite sur un dépôt GitHub avec un `README` et un `.gitignore` Java correct (`target/` exclu).
2. Activez la **protection de branche** sur `main` : PR obligatoire + 1 approbation + status checks requis.
3. Créez une **Issue** « Ajouter un endpoint /menu », ouvrez une branche, faites une **PR** qui la référence avec `Closes #1` .
4. Constatez que vous **ne pouvez pas** pousser directement sur `main` , puis fusionnez via la PR.
5. Créez un **tag** `v1.0.0` et une **release**.

Objectif : un dépôt où `main` est protégée et où tout changement passe par une PR relue.

Atelier GitHub Actions — CI/CD de bout en bout

On automatise enfin tout : à chaque push, **tester** ; sur la branche `main` , **construire et publier** l'image ; sur un tag, **déployer**. C'est l'aboutissement concret du Module 2.

A.1 L'anatomie d'un workflow

Un workflow est un fichier YAML dans `.github/workflows/` . Le vocabulaire :

Terme	Définition
Workflow	le fichier complet, déclenché par un ou plusieurs événements
Event (<code>on</code>)	ce qui le déclenche : <code>push</code> , <code>pull_request</code> , <code>tag</code> , manuel, planifié
Job	un groupe d'étapes qui s'exécute sur une machine ; les jobs tournent en parallèle par défaut
Runner	la machine qui exécute le job (<code>ubuntu-latest</code> fourni par GitHub)
Step	une étape : soit une commande (<code>run</code>), soit une action réutilisable (<code>uses</code>)
Action	un composant réutilisable publié par la communauté (ex. <code>actions/checkout</code>)

A.2 Le pipeline CI : tester QuickBite à chaque push

Créez `.github/workflows/ci.yml` . Comme QuickBite est en **Maven/Java 21**, on s'appuie sur l'action `setup-java` avec cache Maven intégré :

```

name: CI
on:
  push:
    branches: [ main ]
  pull_request:
    # se déclenche aussi sur chaque PR

jobs:
  build-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4          # 1) récupérer le code
      - uses: actions/setup-java@v4        # 2) installer le JDK 21
        with:
          distribution: temurin
          java-version: '21'
          cache: maven                    # met en cache ~/.m2 (build plus rapide)
      - run: mvn -B verify                 # 3) compiler + lancer TOUS les tests
      - uses: actions/upload-artifact@v4   # 4) conserver le rapport de tests
        if: always()                      # même si l'étape précédente a échoué
        with:
          name: test-reports
          path: target/surefire-reports/

```

`mvn -B verify` compile, lance les tests unitaires **et** d'intégration, et échoue (code $\neq$ 0) au premier test rouge — ce qui marque le job en **rouge**. Couplé à la protection de branche (atelier précédent), cela **bloque** la fusion d'un code cassé.

A.3 Matrice : tester sur plusieurs versions

```

jobs:
  test:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        java: [ '17', '21' ]          # exécute le job une fois par version
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: { distribution: temurin, java-version: '${{ matrix.java }}', cache: maven }
      - run: mvn -B verify

```

La **matrice** lance le même job en parallèle pour chaque combinaison — utile pour garantir la compatibilité sur plusieurs versions de langage ou d'OS.

A.4 Construire et publier l'image dans GHCR

Sur la branche `main`, on construit l'image Docker et on la **pousse dans GitHub Container Registry** (`ghcr.io`), gratuit et intégré. Le `GITHUB_TOKEN` est fourni automatiquement :

```

build-push:
  needs: build-test                                # ne démarre QUE si les tests passent
  if: github.ref == 'refs/heads/main' # uniquement sur main
  runs-on: ubuntu-latest
  permissions:
    contents: read
    packages: write                                # autorise l'écriture dans GHCR
  steps:
    - uses: actions/checkout@v4
    - uses: docker/login-action@v3
      with:
        registry: ghcr.io
        username: ${{ github.actor }}
        password: ${{ secrets.GITHUB_TOKEN }}
    - uses: docker/metadata-action@v5 # génère des tags propres (sha, latest...)
      id: meta
      with:
        images: ghcr.io/${{ github.repository }}
    - uses: docker/build-push-action@v6
      with:
        context: .
        push: true
        tags: ${{ steps.meta.outputs.tags }}
        cache-from: type=gha # réutiliser le cache de build entre exécutions
        cache-to: type=gha,mode=max

```

`needs: build-test` exprime la **dépendance** : on ne publie une image que si elle a passé les tests. Le `if:` restreint ce job à `main` (on ne publie pas depuis une PR).

A.5 Le job de déploiement, avec approbation

```

deploy:
  needs: build-push
  runs-on: ubuntu-latest
  environment: production # ★ déclenche l'approbation manuelle configurée sur cet env
  steps:
    - uses: actions/checkout@v4
    - name: Déployer sur Kubernetes
      run: |
        echo "${{ secrets.KUBECONFIG }}" > kubeconfig
        export KUBECONFIG=$PWD/kubeconfig
        kubectl set image deploy/quickbite-api \
          api=ghcr.io/${{ github.repository }}:${{ github.sha }} -n prod
        kubectl rollout status deploy/quickbite-api -n prod

```

En liant le job à l'`environment: production` (configuré en G.5 pour exiger une approbation), GitHub **met le déploiement en pause** jusqu'à ce qu'un humain clique sur « Approve ». C'est la frontière nette entre **livraison continue** (déploiement validé manuellement) et **déploiement continu** (entièrement automatique) du chapitre 9.

A.6 Secrets, variables et bonnes pratiques

- **Secrets** : `${{ secrets.MON_SECRET }}`, définis dans `Settings → Secrets`. Masqués dans les logs, jamais en clair dans le YAML.

- **Permissions minimales** : commencez par `permissions: { contents: read }` au niveau du workflow, et n'élargissez (`packages: write` ...) que par job, là où c'est nécessaire — principe du moindre privilège.
- **Épinglez vos actions** : `actions/checkout@v4` (ou mieux, un SHA) plutôt que `@main`, pour la reproductibilité et la sécurité.
- **concurrency** : annule les exécutions en double sur une même branche.

```
permissions:
  contents: read
concurrency:
  group: ${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: true          # annule l'ancien run si on repousse
```

A.7 Déclencheurs au-delà du push

```
on:
  push:
    tags: [ 'v*' ]                # sur un tag de version (release)
  workflow_dispatch:              # bouton "Run workflow" manuel dans l'UI
  schedule:
    - cron: '0 3 * * 1'          # tous les lundis à 3h (ex. scan de sécurité)
```

💡 La stratégie CI/CD complète de QuickBite

On combine tout en une chaîne lisible : **sur chaque PR** → tests (rien d'autre, on ne publie pas) ; **sur push vers main** → tests puis build & push de l'image taguée par le SHA du commit ; **sur un tag v*** → déploiement en production après approbation manuelle. Chaque étage ne s'exécute que si le précédent est vert (`needs`), et la branche `main` protégée garantit que rien ne fusionne sans CI verte. C'est exactement la définition de DORA : petits changements, fréquents, sûrs.

🔧 Exercice — Atelier final : du commit à la production

Sur votre dépôt QuickBite, assemblez le pipeline complet :

1. **CI** : workflow `ci.yml` qui lance `mvn -B verify` sur chaque PR et push. Cassez un test, ouvrez une PR, et constatez la fusion **bloquée**.
2. **Build & push** : sur `main`, construisez l'image et publiez-la dans `ghcr.io`. Vérifiez qu'elle apparaît dans l'onglet « Packages ».
3. **Cache** : confirmez que le second run est nettement plus rapide grâce au cache Maven et au cache de build Docker.
4. **Environnement protégé** : créez un environnement `production` avec approbation requise, et un job `deploy` qui s'y rattache.
5. **Release** : poussez un tag `v1.1.0` et observez la chaîne se dérouler jusqu'à l'attente d'approbation.

Démonstration finale : racontez le trajet complet d'une modification de code — du `git commit` jusqu'à l'image déployée — en nommant, à chaque étape, l'outil et le garde-fou (test, revue, branche protégée, approbation) qui sécurise le passage. C'est la synthèse de tout le Module 2.

Récapitulatif de la partie pratique

Vous avez maintenant manipulé, de vos mains, toute la chaîne : **Linux** est le terrain (terminal, fichiers, processus, services, scripts) ; **Docker** empaquète QuickBite dans une image légère et reproductible ; **Kubernetes** l'exécute de façon résiliente, scalable et auto-réparatrice ; **GitHub** structure la collaboration (branches, PR, revues, branche protégée) ; et **GitHub Actions** automatise tout, du test à la mise en production sous contrôle. Aucun de ces outils n'a de valeur isolément : leur force vient de la **chaîne** qu'ils forment, au service du même objectif que le Module 1 — livrer de la valeur, vite et sûrement.

Dans le Module 3, nous nous attaquons à une exigence non négociable de tout produit livré : sa **sécurité**.

MODULE 3 — Spring Security

Chapitre 15 — Les fondamentaux de la sécurité applicative

Avant d'écrire la moindre ligne de Spring Security, il faut maîtriser un socle de concepts. Sans eux, on configure des choses sans comprendre ce qu'on protège, et on crée des failles.

15.1 Authentification et autorisation : deux choses distinctes

Ce sont les deux piliers de la sécurité, et on les confond très souvent. Pourtant, ils répondent à deux questions différentes :

- L'**authentification** (en abrégé *AuthN*) répond à : « **Qui es-tu ?** » On vérifie l'**identité** : tu prétends être Alice ; prouve-le (avec un mot de passe, un token, un certificat...).
- L'**autorisation** (en abrégé *AuthZ*) répond à : « **As-tu le droit de faire cela ?** » Une fois qu'on sait que tu es Alice, a-t-elle la permission d'accéder à la console d'administration ?

L'ordre est immuable : on authentifie **d'abord**, on autorise **ensuite**.

	Authentification (AuthN)	Autorisation (AuthZ)
Question posée	« Qui es-tu ? »	« As-tu le droit ? »
Vérifie	L'identité	Les permissions
Vient	En premier	Après
Exemple QuickBite	Se connecter à l'application	Accéder à la console admin

! Une erreur de raisonnement dangereuse

Un utilisateur **authentifié n'est pas pour autant autorisé**. Le fait de savoir « c'est bien Alice » ne dit rien sur ce qu'Alice a le droit de faire. Les deux contrôles sont indépendants et tous les deux obligatoires. Oublier le second (l'autorisation) est la cause de la faille la plus répandue du web : le « *Broken Access Control* », numéro 1 du classement OWASP.

15.2 Stateful ou stateless : deux façons de « se souvenir » de l'utilisateur

Une fois l'utilisateur connecté, comment le serveur se souvient-il de lui à la requête suivante ? Deux approches opposées :

- **Stateful (avec session)**. Le serveur crée une **session** en mémoire et donne au client un identifiant de session (un cookie `JSESSIONID`). À chaque requête, le client renvoie ce cookie, et le serveur retrouve la session. Simple, mais le serveur doit **garder un état** : si on a plusieurs serveurs, il faut partager les sessions entre eux, ce qui complique la mise à l'échelle.

- **Stateless (avec token).** Le serveur ne garde **rien** en mémoire. Il remet au client un **token** (typiquement un JWT, qu'on verra au chapitre 18) qui contient lui-même les informations d'identité, signées. À chaque requête, le client présente ce token, et le serveur le vérifie sans rien avoir à stocker.

Critère	Stateful (session)	Stateless (token)
Stockage côté serveur	Oui (les sessions)	Aucun
Mise à l'échelle horizontale	Difficile	Facile
Révocation immédiate	Facile (on invalide la session)	Difficile (le token reste valide jusqu'à expiration)
Cas d'usage idéal	Application web classique	API REST, application mobile, microservices

Pour une **API REST** moderne (notre cas avec QuickBite), on choisit presque toujours le **stateless**, car il s'adapte naturellement à la montée en charge et aux clients variés (web, mobile).

15.3 Stocker les mots de passe : le hachage

Voici une règle absolue : **on ne stocke jamais un mot de passe en clair**, ni même chiffré de façon réversible. On stocke un **hachage** (*hash*) : le résultat d'une fonction mathématique à sens unique. À la connexion, on hache le mot de passe saisi et on compare au hachage stocké. Ainsi, même si la base de données est volée, les mots de passe ne sont pas directement exposés.

Mais tous les hachages ne se valent pas. Pour les mots de passe, il faut une fonction **lente** et **salée** :

- Le **sel** (*salt*) est une valeur aléatoire, différente pour chaque mot de passe, ajoutée avant le hachage. Il empêche les attaques par tables précalculées (*rainbow tables*).
- La **lenteur** est volontaire : elle ralentit l'attaquant qui essaierait des millions de mots de passe (force brute), sans gêner l'utilisateur légitime qui ne se connecte qu'une fois.

```
@Bean
public PasswordEncoder passwordEncoder() {
    // BCrypt avec un "coût" (work factor) de 12 :
    // plus le coût est élevé, plus le hachage est lent (donc sûr)
    return new BCryptPasswordEncoder(12);
}
```


Algorithme	Caractéristique
BCrypt	Standard éprouvé, intègre le sel, coût ajustable. Le choix par défaut sûr.
Argon2	Lauréat du concours mondial de hachage de mots de passe ; résistant aux attaques par GPU. Recommandé pour les nouveaux projets.
PBKDF2	Largement supporté, conforme aux normes FIPS.
~~MD5, SHA-1, SHA-256 seuls~~	À proscrire pour les mots de passe : ils sont conçus pour être rapides, donc faciles à attaquer par force brute.

⚠ « Mais SHA-256 est réputé sûr, non ? »

SHA-256 est un excellent hachage... pour vérifier l'intégrité d'un fichier. Mais pour un mot de passe, sa **rapidité** est un défaut : un attaquant peut tester des milliards de combinaisons par seconde. BCrypt et Argon2 sont **volontairement lents**, ce qui rend la force brute économiquement impossible. La vitesse, qualité ailleurs, est un défaut ici.

15.4 Les attaques courantes et leurs parades

Attaque	En quoi elle consiste	La parade
Force brute	Essayer des milliers de mots de passe jusqu'à trouver	Hachage lent, limitation du débit, blocage du compte, double authentification
MITM (homme du milieu)	Intercepter le trafic entre client et serveur	HTTPS/TLS partout , en-tête HSTS
XSS	Injecter du JavaScript malveillant dans une page	Échapper les sorties, politique CSP, cookies HttpOnly
CSRF	Forcer le navigateur d'une victime à envoyer une requête à son insu	Token anti-CSRF, attribut de cookie SameSite , ou API stateless
Injection SQL	Glisser du code SQL dans un champ de saisie	Requêtes paramétrées, utilisation d'un ORM

Chapitre 16 — L'architecture de Spring Security 6

16.1 L'idée centrale : une chaîne de filtres

Spring Security ne s'éparpille pas dans tout votre code. Il s'insère **avant** votre application, sous la forme d'une **chaîne de filtres** (`SecurityFilterChain`). Chaque requête HTTP entrante traverse cette chaîne **avant** d'atteindre votre contrôleur. Chaque filtre a un rôle : l'un extrait les identifiants, un autre vérifie l'autorisation, etc.



16.2 Les composants, un par un

Composant	Son rôle
<code>SecurityFilterChain</code>	Déclare les règles : quelles URL sont publiques, lesquelles sont protégées, comment on se connecte
<code>AuthenticationManager</code>	Le chef d'orchestre de l'authentification
<code>AuthenticationProvider</code>	Celui qui vérifie réellement les identifiants
<code>UserDetailsService</code>	Celui qui va chercher l'utilisateur (par exemple dans la base de données)
<code>UserDetails</code> / <code>GrantedAuthority</code>	La représentation de l'utilisateur et de ses rôles/permissions
<code>SecurityContext</code>	Le « porte-document » qui conserve l'utilisateur authentifié pendant toute la durée de la requête

Le flux se lit ainsi : la requête arrive, les filtres en extraient les identifiants, l'`AuthenticationManager` délègue à un `AuthenticationProvider` qui, via le `UserDetailsService`, charge l'utilisateur et vérifie son mot de passe. Si tout est bon, un objet `Authentication` (contenant l'identité et les rôles) est placé dans le `SecurityContext`. Ensuite, pour chaque ressource demandée, l'`AuthorizationManager` vérifie que cet utilisateur a bien le droit d'y accéder.

16.3 Le grand changement de Spring Security 6

! `WebSecurityConfigurerAdapter` n'existe plus

Si vous trouvez sur Internet du code qui étend `WebSecurityConfigurerAdapter` et surcharge une méthode `configure(HttpSecurity http)`, c'est de l'**ancien code (Spring Security 5)**. Cette classe a été **supprimée** en Spring Security 6. Le nouveau modèle consiste à déclarer un **bean** de type `SecurityFilterChain` et à configurer la sécurité avec une **syntaxe à base de lambdas**. Beaucoup de tutoriels obsolètes traînent encore : vérifiez toujours la version.

Voici la configuration **moderne**, telle qu'on l'écrit aujourd'hui :

```

@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/", "/public/**").permitAll() // accès libre
                .requestMatchers("/admin/**").hasRole("ADMIN") // réservé aux admins
                .anyRequest().authenticated() // tout le reste : connecté
            )
            .httpBasic(Customizer.withDefaults()) // active l'authentification Basic
            .formLogin(Customizer.withDefaults()); // active le formulaire de login
        return http.build();
    }

    @Bean
    PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }
}

```

Lisez la section `authorizeHttpRequests` de haut en bas, comme une liste de règles évaluées dans l'ordre : les URL publiques d'abord, puis les URL d'admin réservées au rôle ADMIN, et enfin une règle « attrape-tout » qui exige au minimum d'être authentifié.

Exercice — Atelier 1 : premier projet sécurisé

1. Générez un projet Spring Boot 3 (dépendances Web + Security) sur start.spring.io. Lancez-le et observez : Spring Security protège **tout** par défaut et génère un mot de passe affiché dans la console.
2. Créez un contrôleur avec une route publique `/` et une route protégée `/private`, puis configurez la `SecurityFilterChain`.
3. Définissez deux utilisateurs en mémoire (`user` et `admin`) avec des rôles différents, et testez l'accès à `/admin`.

```

@Bean
UserDetailsService users(PasswordEncoder encoder) {
    UserDetails user = User.withUsername("user")
        .password(encoder.encode("user123")).roles("USER").build();
    UserDetails admin = User.withUsername("admin")
        .password(encoder.encode("admin123")).roles("ADMIN").build();
    return new InMemoryUserDetailsManager(user, admin);
}

```

Chapitre 17 — Authentification et autorisation (RBAC)

On passe maintenant des utilisateurs « en mémoire » (pratiques pour démarrer) à de vrais utilisateurs persistés en base, et on met en place un contrôle d'accès fin.

17.1 Charger les utilisateurs depuis la base

On crée une entité JPA pour représenter un utilisateur :

```
@Entity
@Table(name = "users")
public class UserEntity {
    @Id @GeneratedValue
    private Long id;
    @Column(unique = true, nullable = false)
    private String username;
    @Column(nullable = false)
    private String password; // le HASH BCrypt, jamais le mot de passe en clair
    @ManyToMany(fetch = FetchType.EAGER)
    private Set<RoleEntity> roles = new HashSet<>();
    // getters / setters
}
```

Puis on implémente un `UserDetailsService` qui va chercher cet utilisateur et le traduit dans le format attendu par Spring Security :

```
@Service
public class JpaUserDetailsService implements UserDetailsService {

    private final UserRepository repo;
    public JpaUserDetailsService(UserRepository repo) { this.repo = repo; }

    @Override
    public UserDetails loadUserByUsername(String username) {
        UserEntity u = repo.findByUsername(username)
            .orElseThrow(() -> new UsernameNotFoundException(username));
        // on transforme les rôles en "autorités" comprises par Spring
        var authorities = u.getRoles().stream()
            .map(r -> new SimpleGrantedAuthority("ROLE_" + r.getName()))
            .collect(Collectors.toList());
        return new org.springframework.security.core.userdetails.User(
            u.getUsername(), u.getPassword(), authorities);
    }
}
```

Message d'erreur volontairement vague

Lorsqu'une connexion échoue, renvoyez toujours un message **générique** du type « identifiants invalides », sans préciser si c'est le nom d'utilisateur ou le mot de passe qui est faux. Sinon, un attaquant peut **énumérer les comptes existants** (« ce login existe, mais pas ce mot de passe »). Ne donnez jamais d'information utile à l'attaquant.

17.2 Rôles, autorités, permissions

Distinguons deux niveaux de granularité :

- Un **rôle** est un regroupement large : `ROLE_ADMIN`, `ROLE_USER`. C'est grossier.
- Une **autorité** ou **permission** est un droit précis : `order:read` (lire les commandes), `order:cancel` (annuler une commande). C'est fin.

Un modèle **RBAC** mature associe : *un utilisateur a des rôles, et chaque rôle regroupe des permissions*. On contrôle ensuite les accès par **permission** plutôt que par rôle. Pourquoi ? Parce que si demain on crée un nouveau rôle « Manager » qui peut lire mais pas annuler les commandes, on lui attribue simplement la permission `order:read` — sans toucher au code, qui vérifie des permissions et non des rôles.

```
Utilisateur —*..*— Rôle —*..*— Permission
alice          ADMIN      order:read, order:cancel, user:manage
bob           CLIENT      order:read, order:create
```

17.3 Protéger les méthodes

On active la sécurité au niveau des méthodes :

```
@Configuration
@EnableMethodSecurity // active @PreAuthorize, @PostAuthorize, etc.
public class MethodSecurityConfig { }
```

Puis on annote directement les méthodes des contrôleurs :

```
@RestController
@RequestMapping("/orders")
public class OrderController {

    @GetMapping
    @PreAuthorize("hasAuthority('order:read')") // exige la permission de lecture
    public List<OrderDto> all() { ... }

    @DeleteMapping("/{id}")
    @PreAuthorize("hasRole('ADMIN')") // exige le rôle ADMIN
    public void delete(@PathVariable Long id) { ... }

    // Expression plus fine : un admin OU le propriétaire de la ressource
    @GetMapping("/{id}")
    @PreAuthorize("hasRole('ADMIN') or #username == authentication.name")
    public OrderDto getOne(@PathVariable Long id,
                           @RequestParam String username) { ... }
}
```

Annotation	Quand elle est évaluée	Remarque
<code>@PreAuthorize</code>	Avant l'exécution de la méthode	La plus utilisée ; accepte des expressions riches
<code>@PostAuthorize</code>	Après , sur la valeur de retour	Pour filtrer selon l'objet renvoyé
<code>@Secured</code>	Avant	Plus ancien, ne gère que les rôles

⚠ Le piège du préfixe `ROLE_`

En Spring Security, `hasRole('ADMIN')` cherche en réalité une autorité nommée `ROLE_ADMIN` : le préfixe `ROLE_` est ajouté automatiquement. Si vos rôles en base sont déjà préfixés (ou pas), c'est une source de bugs très fréquente où l'accès est refusé sans raison apparente. Choisissez une convention et tenez-vous-y.

Exercice — Atelier 2 : modèle rôles/permissions

1. Créez les entités `UserEntity` et `RoleEntity` (avec permissions) et leurs repositories ; insérez un admin et un client.
2. Implémentez le `JpaUserDetailsService`.
3. Protégez `/orders` (lecture pour CLIENT), `/admin/**` (ADMIN), et la suppression (ADMIN uniquement).
4. Testez avec Postman et vérifiez bien les codes de réponse : **200** (autorisé), **401** (non authentifié), **403** (authentifié mais pas le droit).

401 contre 403 : la nuance à retenir

Ces deux codes d'erreur sont souvent confondus. **401 Unauthorized** signifie « je ne sais pas qui tu es » (tu n'es pas authentifié, ou ton token est invalide). **403 Forbidden** signifie « je sais qui tu es, mais tu n'as pas le droit ». Le 401 concerne l'authentification, le 403 concerne l'autorisation.

Chapitre 18 — Les tokens JWT en détail

18.1 Ce qu'est un JWT et pourquoi il est pratique

Un **JWT** (*JSON Web Token*) est un jeton **auto-porteur** : il contient lui-même les informations d'identité de l'utilisateur, et il est **signé** numériquement. Grâce à la signature, le serveur peut vérifier que le token est authentique **sans interroger la base de données** à chaque requête. C'est exactement ce qu'il faut pour le **stateless** vu au chapitre 15.

Un JWT se compose de **trois parties**, encodées et séparées par des points :

```
header.payload.signature
```

```
eyJhbGciOiJIUzI1NiJ9 . eyJzdWIiOiJhbGljZSIsInJvbGUiOiJBRElJTjJ9 . 3xT9...signature
```

Partie	Contenu	Exemple
Header	L'algorithme de signature et le type	<code>{"alg":"HS256","typ":"JWT"}</code>
Payload	Les <i>claims</i> : identité, rôles, date d'expiration...	<code>{"sub":"alice","role":"ADMIN","exp":...}</code>
Signature	La signature cryptographique du header + payload	garantit l' intégrité

! Un JWT n'est PAS chiffré — il est seulement signé

C'est le malentendu le plus dangereux. Le payload d'un JWT est seulement **encodé** en base64 : n'importe qui peut le décoder et **lire son contenu** (essayez sur le site jwt.io). La signature garantit qu'il n'a pas été **modifié**, mais **pas** qu'il est secret. Conséquence : ne mettez **jamais** de donnée sensible (mot de passe, numéro de carte, secret) dans un JWT.

18.2 Access token et refresh token

Un seul token poserait un dilemme : s'il dure longtemps, le risque est grand en cas de vol ; s'il dure peu, l'utilisateur doit se reconnecter sans cesse. La solution est d'utiliser **deux tokens** :

- L'**access token** a une durée **courte** (5 à 15 minutes) et accompagne chaque requête. S'il est volé, le risque est limité dans le temps.
- Le **refresh token** a une durée **longue** (plusieurs jours), est stocké de façon sécurisée, et ne sert qu'à **une seule chose** : obtenir un nouvel access token quand l'ancien expire.

```
Connexion → access token (15 min) + refresh token (7 jours)
|
├─ requêtes à l'API avec l'access token
|
└─ access token expiré → POST /auth/refresh (avec le refresh token)
                        |
                        └─ nouvel access token
```

⚠ Rotation et stockage des refresh tokens

Deux bonnes pratiques essentielles : (1) appliquez la **rotation** — chaque fois qu'un refresh token est utilisé, il est invalidé et remplacé par un nouveau, ce qui permet de détecter un vol. (2) Stockez les refresh tokens en base pour pouvoir les **révoquer**. Côté navigateur, préférez un cookie `HttpOnly` + `SameSite` plutôt que le `localStorage`, qui est vulnérable aux attaques XSS.

18.3 Implémenter JWT dans Spring Security 6

Il existe deux approches. L'approche « **maison** » (un filtre que l'on écrit soi-même) est excellente pour **comprendre la mécanique** ; l'approche « **resource server** » standard de Spring est celle qu'on privilégie en production car elle demande très peu de code. Présentons d'abord l'approche maison.

Le service qui fabrique et lit les tokens (avec la bibliothèque JJWT) :

```

@Service
public class JwtService {
    private final SecretKey key = Keys.hmacShaKeyFor(
        System.getenv("JWT_SECRET").getBytes(StandardCharsets.UTF_8));

    public String generateAccess(UserDetails user) {
        return Jwts.builder()
            .subject(user.getUsername())
            .claim("roles", user.getAuthorities().stream()
                .map(GrantedAuthority::getAuthority).toList())
            .issuedAt(new Date())
            .expiration(new Date(System.currentTimeMillis() + 900_000)) // 15 minutes
            .signWith(key)
            .compact();
    }

    public Jws<Claims> parse(String token) {
        return Jwts.parser().verifyWith(key).build().parseSignedClaims(token);
    }
}

```

Le filtre qui, à chaque requête, lit le token et place l'utilisateur dans le contexte de sécurité :

```

@Component
public class JwtAuthFilter extends OncePerRequestFilter {

    private final JwtService jwt;
    public JwtAuthFilter(JwtService jwt) { this.jwt = jwt; }

    @Override
    protected void doFilterInternal(HttpServletRequest req,
        HttpServletResponse res, FilterChain chain)
        throws ServletException, IOException {
        String header = req.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            try {
                Claims claims = jwt.parse(header.substring(7)).getPayload();
                var roles = ((List<?>) claims.get("roles")).stream()
                    .map(r -> new SimpleGrantedAuthority(r.toString())).toList();
                var auth = new UsernamePasswordAuthenticationToken(
                    claims.getSubject(), null, roles);
                SecurityContextHolder.getContext().setAuthentication(auth);
            } catch (JwtException e) {
                SecurityContextHolder.clearContext(); // token invalide → non authentifié
            }
        }
        chain.doFilter(req, res);
    }
}

```

La configuration stateless, qui désactive les sessions et insère notre filtre :


```
@Bean
SecurityFilterChain api(HttpSecurity http, JwtAuthFilter jwtFilter) throws Exception {
    http
        .csrf(csrf -> csrf.disable()) // API stateless → CSRF non pertinent (voir ch. 20)
        .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/auth/**").permitAll()
            .requestMatchers("/admin/**").hasRole("ADMIN")
            .anyRequest().authenticated())
        .addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class);
    return http.build();
}
```

L'**approche standard** (resource server), pour comparaison, se résume à quelques lignes : Spring valide alors lui-même les tokens à partir d'une URL de clés publiques (JWKS) fournie par le serveur d'autorisation.

```
http.oauth2ResourceServer(oauth2 -> oauth2.jwt(Customizer.withDefaults()));
```

18.4 Gérer proprement les erreurs

On distingue les deux cas vus au chapitre 17 :

```
// 401 : l'utilisateur n'est pas authentifié
@Component
public class JwtEntryPoint implements AuthenticationEntryPoint {
    public void commence(HttpServletRequest req, HttpServletResponse res,
        AuthenticationException ex) throws IOException {
        res.sendError(HttpServletResponse.SC_UNAUTHORIZED, "Authentification requise");
    }
}

// 403 : l'utilisateur est authentifié mais n'a pas le droit
@Component
public class RestAccessDeniedHandler implements AccessDeniedHandler {
    public void handle(HttpServletRequest req, HttpServletResponse res,
        AccessDeniedException ex) throws IOException {
        res.sendError(HttpServletResponse.SC_FORBIDDEN, "Accès refusé");
    }
}
```

Exercice — Atelier 3 : API REST sécurisée par JWT

Implémentez les endpoints suivants : `POST /auth/login` (renvoie access + refresh), `POST /auth/refresh` (renvoie un nouvel access), `GET /users/me` (authentifié) et `GET /admin/users` (réservé ADMIN).

Critères de réussite : la connexion renvoie deux tokens ; `/users/me` fonctionne avec le Bearer ; `/admin/users` renvoie 403 pour un non-admin ; un token expiré renvoie un 401 propre, et le refresh fournit un nouvel access token.

Pièges à éviter : oublier `STATELESS` (Spring recrée alors des sessions), placer le filtre au mauvais endroit, ou utiliser un secret trop court pour HS256 (il faut au moins 256 bits).

Chapitre 19 — OAuth2, OpenID Connect et Keycloak

19.1 Le problème : déléguer l'authentification

Jusqu'ici, notre application gérait elle-même les mots de passe. Mais souvent, on veut :

- permettre la connexion « avec Google » ou « avec GitHub » (sans gérer de mot de passe) ;
- centraliser l'authentification de plusieurs applications (le *Single Sign-On*, ou SSO) ;
- qu'une application puisse accéder à mes données sur un autre service, **sans lui donner mon mot de passe**.

C'est exactement ce que résolvent OAuth2 et OpenID Connect.

19.2 OAuth2 et OpenID Connect : la différence

- **OAuth2** est un protocole d'**autorisation déléguée**. Il répond à : « cette application a-t-elle le droit d'accéder à telles ressources en mon nom ? » Il n'a pas été conçu, à l'origine, pour dire **qui** est l'utilisateur.
- **OpenID Connect (OIDC)** est une **couche d'authentification** ajoutée par-dessus OAuth2. Il répond à : « **qui** est l'utilisateur ? » et ajoute pour cela un **ID token**.

En résumé : OAuth2 gère l'**autorisation**, OIDC ajoute l'**authentification**. Aujourd'hui, quand on parle de « se connecter avec Google », on utilise OIDC.

19.3 Les acteurs

Rôle	Description
Resource Owner	L'utilisateur (le propriétaire des données)
Client	L'application qui veut accéder aux ressources (notre front, par exemple)
Authorization Server	Le serveur qui authentifie et délivre les tokens (Keycloak, Google...)
Resource Server	L'API qui héberge les ressources protégées (notre QuickBite API)

19.4 Le flow recommandé : Authorization Code + PKCE

Il existe plusieurs « flows » (scénarios d'échange). Le standard actuel pour le web et le mobile est l'**Authorization Code Flow**, renforcé par **PKCE** :

```

Utilisateur → Client → Authorization Server (affiche la page de connexion)
                        | (l'utilisateur s'authentifie et consent)
                        ← code d'autorisation ┘
Client → échange ce code (+ son secret PKCE) → Authorization Server
                        ← access token (+ id token + refresh) ┘
Client → appelle l'API avec l'access token → Resource Server
  
```

Flow	Usage	Statut
Authorization Code + PKCE	Web, mobile, applications monopage (SPA)	Recommandé
Client Credentials	De machine à machine (pas d'utilisateur)	OK pour les services
~~Implicit~~	Ancien flow pour SPA	Déconseillé (remplacé par Code+PKCE)
~~Password (ROPC)~~	L'app récupère directement le mot de passe	Déconseillé

■ À quoi sert PKCE ?

PKCE (*Proof Key for Code Exchange*) protège les clients « publics » (applications mobiles ou monopages) qui ne peuvent pas garder un secret. Le principe : au début, le client invente un secret aléatoire (`code_verifier`) et n'envoie que son empreinte (`code_challenge`). À la fin, il prouve qu'il détient bien le secret original. Ainsi, même si un attaquant intercepte le code d'autorisation, il ne peut pas l'échanger sans le secret.

19.5 ID token contre access token

Une distinction subtile mais importante en OIDC :

- L'**ID token** prouve **qui est l'utilisateur** (il contient son nom, son e-mail...). Il est destiné au **client** (l'application front).
- L'**access token** autorise l'accès à une **API**. Il est destiné au **resource server** (l'API).

19.6 Keycloak : le serveur d'identité

Keycloak est un serveur open source de gestion des identités et des accès (IAM). Il fait office d'*Authorization Server* : il gère les utilisateurs, l'authentification, le SSO, les rôles, et délivre les tokens. On le lance facilement avec Docker :

```
docker run -p 8080:8080 \
  -e KEYCLOAK_ADMIN=admin -e KEYCLOAK_ADMIN_PASSWORD=admin \
  quay.io/keycloak/keycloak:24.0 start-dev
```

Concept Keycloak	Définition
Realm	Un espace totalement isolé regroupant ses propres utilisateurs, clients et rôles
Client	Une application enregistrée (un front, une API...)
Roles	Les rôles, définis au niveau du realm ou d'un client
Mappers	Ce qui injecte des données (rôles, attributs) dans les tokens

Côté Spring, on configure alors l'application **soit en client** (pour gérer le login), **soit en resource server** (pour valider les tokens émis par Keycloak) :

```
# L'API en resource server : elle valide les tokens émis par Keycloak
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          issuer-uri: http://localhost:8080/realms/quickbite
```

Mapper les rôles Keycloak

Keycloak place les rôles dans une partie du token nommée `realm_access.roles` (ou `resource_access`). Par défaut, Spring ne sait pas les y trouver. Vous devez écrire un petit « convertisseur » qui extrait ces rôles et les transforme en autorités préfixées `ROLE_`, pour que `hasRole()` fonctionne. C'est la cause n°1 des « 403 inexplicables » avec Keycloak.

Exercice — Atelier 4 : login OAuth2 et front protégé

1. Lancez Keycloak (Docker), créez un realm `quickbite`, un client, des rôles (`admin`, `client`) et deux utilisateurs.
2. Configurez l'API en resource server et mappez les rôles Keycloak.
3. Obtenez un token via Keycloak (avec Postman) et appelez `/admin/**` pour vérifier l'autorisation.
4. Configurez un front (Angular ou React) avec une bibliothèque OIDC, en Authorization Code + PKCE ; le front appelle l'API avec le token et affiche ou masque la console admin selon le rôle.

Astuce de debug : collez le token sur jwt.io pour vérifier les claims et les rôles. C'est extrêmement révélateur.

Chapitre 20 — Durcissement et bonnes pratiques (OWASP)

On a une application qui authentifie et autorise. Reste à la **durcir** contre le monde réel.

20.1 CORS et CSRF : deux notions qu'on confond toujours

Malgré leurs noms proches, ce sont deux choses **totalelement différentes** :

	CORS	CSRF
Ce que c'est	Une permission : autoriser un site d'un autre domaine à appeler votre API depuis un navigateur	Une attaque (et sa parade) : empêcher qu'une requête soit forgée à l'insu de l'utilisateur
Nature	Un mécanisme de sécurité du navigateur	Une vulnérabilité
Concerne	Les API appelées par un front hébergé ailleurs	Les sessions et les cookies

```
@Bean
CorsConfigurationSource corsSource() {
    CorsConfiguration c = new CorsConfiguration();
    c.setAllowedOrigins(List.of("https://app.quickbite.com")); // qui a le droit d'appeler
    c.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE"));
    c.setAllowedHeaders(List.of("Authorization", "Content-Type"));
    UrlBasedCorsConfigurationSource s = new UrlBasedCorsConfigurationSource();
    s.registerCorsConfiguration("/**", c);
    return s;
}
```

⚠ Quand désactiver CSRF (et quand surtout pas)

On voit beaucoup de code qui désactive CSRF sans réfléchir. La règle : pour une API **stateless** authentifiée par token (sans cookie de session), CSRF n'est pas pertinent, on peut le désactiver. Mais pour une application web classique **avec session et cookies**, CSRF doit **impérativement rester activé**. Ne désactivez jamais CSRF « par habitude » ou pour faire taire une erreur.

20.2 Limiter le débit (rate limiting)

Pour contrer la force brute et les abus, on **limite le nombre de requêtes** par client ou par adresse IP sur une période donnée. En Spring, on peut utiliser la bibliothèque **Bucket4j**, ou mieux, traiter cela en amont au niveau de la **passerelle d'API** (API Gateway) ou du reverse proxy. C'est particulièrement crucial sur l'endpoint de connexion.

20.3 Gérer les exceptions sans fuiter d'information

Une erreur mal gérée peut révéler à l'attaquant des détails internes (structure de la base, chemins de fichiers...). On centralise donc la gestion des erreurs pour **logger le détail côté serveur** mais ne renvoyer au client qu'un **message générique** :

```
@RestControllerAdvice
public class ApiExceptionHandler {
    @ExceptionHandler(Exception.class)
    public ResponseEntity<ApiError> handle(Exception ex) {
        // on loggue le détail en interne, on renvoie un message neutre au client
        return ResponseEntity.status(500).body(new ApiError("Erreur interne"));
    }
}
```

20.4 Audit et journalisation

Tracez les **événements de sécurité** : connexions réussies et échouées, accès refusés. Avec Spring Boot Actuator, exposez ces informations — mais **protégez** ces endpoints, ne les ouvrez pas publiquement. Et une règle absolue : **ne journalisez jamais de secret, de mot de passe ou de token**.

20.5 Architecture microservices : la passerelle d'API

Dans une architecture en microservices, on place une **API Gateway** (par exemple Spring Cloud Gateway) en point d'entrée unique. Elle centralise l'authentification, la limitation de débit, le routage et la terminaison TLS :

```
Client → API Gateway (authentification, rate limit, TLS) → Service A
                                                    ↳ Service B
                                                    ↳ Service C
```

On prévoit aussi la **rotation des clés de signature** des JWT : en gardant plusieurs clés actives identifiées par un `kid`, on peut renouveler régulièrement les clés sans invalider brutalement tous les tokens existants.

20.6 La check-list de durcissement OWASP

L'**OWASP** (*Open Worldwide Application Security Project*) publie le « Top 10 » des risques de sécurité web. Voici une check-list pratique inspirée de leurs recommandations :

- [] HTTPS/TLS partout, HSTS activé, redirection automatique HTTP → HTTPS.
- [] Mots de passe hachés avec BCrypt ou Argon2, et politique de robustesse.
- [] Tokens à durée courte, rotation des refresh tokens, secret de signature fort.
- [] Validation et nettoyage de **toutes** les entrées (contre les injections).
- [] En-têtes de sécurité : CSP, X-Content-Type-Options, X-Frame-Options.
- [] CORS restrictif et CSRF cohérent avec le modèle de session.
- [] Limitation de débit et blocage de compte sur la connexion.

- [] Principe du moindre privilège partout (rôles, permissions, comptes de service).
- [] Dépendances tenues à jour (scan de vulnérabilités), aucun secret en clair.
- [] Journaux d'audit sans données sensibles, supervision des anomalies.

! Le risque numéro un, encore et toujours

Dans le classement OWASP actuel, le risque le plus répandu est le « **Broken Access Control** » — c'est-à-dire une autorisation mal faite ou absente. C'est précisément ce que les chapitres 17 et 18 vous ont appris à éviter. La leçon : la partie la plus importante de la sécurité n'est pas la cryptographie sophistiquée, c'est de **vérifier rigoureusement, à chaque accès, que l'utilisateur a bien le droit**.

Exercice — TP final : un système complet et sécurisé

Livrez un système QuickBite complet intégrant tout le module : une **API REST sécurisée par JWT** (login/refresh, endpoints protégés) ; une **console admin avec RBAC** (rôles et permissions fins) ; une **intégration Keycloak** (authentification et rôles délégués) ; un **front protégé en OAuth2/OIDC** ; et une **documentation Postman** accompagnée d'une note de sécurité (CORS, CSRF, rotation, check-list OWASP renseignée).

20.7 Conclusion du cursus complet

Vous avez parcouru les trois piliers d'un produit logiciel moderne, et ils forment un tout cohérent :

- Le **Module 1 (Agile)** vous a appris à décider **quoi** construire, **dans quel ordre**, et comment apprendre vite des utilisateurs.
- Le **Module 2 (DevOps)** vous a appris à **livrer** ce produit vite et de façon fiable, du code à la production.
- Le **Module 3 (Sécurité)** vous a appris à **protéger** ce qui est livré.

La phrase à emporter : **on construit le bon produit (Agile), on le livre bien (DevOps), et on le protège sérieusement (Sécurité)**. Ces trois disciplines ne sont pas des silos : ce sont trois facettes d'un même métier, celui de livrer de la valeur, durablement et en confiance.